FACULDADE DE TECNOLOGIA DE RIBEIRÃO PRETO - FATEC
CURSO SUPERIOR DE TECNOLOGIA EM
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

EURICO FERREIRA DE SOUZA JÚNIOR

EMOJIGAME:
DESENVOLVIMENTO DE WEB GAME EM MODO TEXTO UNICODE.

Ribeirão Preto
2018

EURICO FERREIRA DE SOUZA JÚNIOR

EMOJIGAME:
DESENVOLVIMENTO DE WEB GAME EM MODO TEXTO UNICODE.

Relatório final, Apresentado à Faculdade de Tecnologia de Ribeirão Preto como parte das exigências para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Área de concentração: Desenvolvimento de jogos

Orientador: Me. Fabrício Gustavo Henrique

RIBEIRÃO PRETO
2018

Souza Júnior, Eurico Ferreira de
Emojigame: Desenvolvimento de web game em modo texto unicode./Eurico Ferreira de Souza Júnior.
Ribeirão Preto: FATEC, 2018.
77 f.

Monografia (Trabalho de Conclusão de Curso)- Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas – Fatec Ribeirão Preto.

1. Software Engineering.2. Game Design. 3. Web Game. 4. Unicode. 5. Emoji. Souza Júnior, Eurico Ferreira de

CDD -- 006.7

EURICO FERREIRA DE SOUZA JÚNIOR

EMOJIGAME:
DESENVOLVIMENTO DE WEB GAME EM MODO TEXTO UNICODE.

Relatório final, Apresentado à Faculdade de Tecnologia de Ribeirão Preto como parte das exigências para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Área de concentração: Desenvolvimento de jogos

Orientador: Me. Fabrício Gustavo Henrique

Ribeirão Preto, ____ de_____de_____

Banca Examinadora

Presidente da banca (Orientador): Me. Fabrício Gustavo Henrique

Fatec _____

Assinatura

Parecer final:_____ Data:_____

Prof.(a). _____

Instituição:_____ Assinatura:_____

Prof.(a). _____

Instituição:_____ Assinatura:_____

Prof.(a). _____

Instituição:_____ Assinatura:_____

Eu gostaria de agradecer em primeiro lugar e principalmente aos cidadãos contribuintes paulistas e ao Governo do Estado de São Paulo por proverem ensino público de qualidade na Faculdade de Tecnologia de Ribeirão Preto.

Agradeço também os ensinamentos, apoio e paciência dos professores, colegas de classe e funcionários da FATEC durante esses três anos de estudos gratificantes e inesquecíveis. Agradeço especialmente o Prof. Fabrício por me orientar na elaboração desse trabalho.

Não poderia deixar de agradecer o apoio dos amigos e familiares que me ajudaram a persistir e completar esse percurso acadêmico.

Por fim agradeço ao leitor, desejando que a leitura deste trabalho possa ser agradável e proveitosa.

“Some people say, "Give the customers what they want." But that's not my approach. Our job is to figure out what they're going to want before they do. I think Henry Ford once said, "If I'd asked customers what they wanted, they would have told me, 'A faster horse!'" People don't know what they want until you show it to them. That's why I never rely on market research. Our task is to read things that are not yet on the page.”

(Steve Jobs)

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

RESUMO

Este trabalho testa a possibilidade do desenvolvimento de *web games* utilizando os emoji e outros símbolos e letras do conjunto unicode como ativos gráficos e indica a viabilidade da proposta através do relatório do projeto e desenvolvimento do protótipo de jogo intitulado EMOJIGAME realizado com Desenvolvimento Ágil de engenharia de software pela primeira turma do curso superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Faculdade de Tecnologia de Ribeirão preto, disponível em emojigame.azurewebsites.net. Contém também o *Game Design Document* e Plano de Negócio propondo a continuação do projeto.

Palavras-chave: Engenharia de software. Desenvolvimento de jogos. Web Game. Unicode. Emoji.

ABSTRACT

This work tests the possibility of the development of web games using the emoji and other symbols and letters of the unicode set as graphic assets and indicates the feasibility of the proposal through the design and development report of the game prototype titled EMOJIGAME, carried out with Agile Development of software engineering by the first group of the higher education course of Technology in Systems Analysis and Development of the Faculty of Technology of Ribeirão Preto, available at emojigame.azurewebsites.net. It also contains the Game Design Document and Business Plan proposing to move forward with the project.

Keywords: Software Engineering. Game Design. Web Game. Unicode. Emoji.

SUMÁRIO

1. **INTRODUÇÃO**…… 8
1.1 Contextualização e Motivação……..……..… 8
1.2 O caso (ou Projeto)…….. 11
1.3 Formulação do Problema (ou Questão da Pesquisa)…….… 12
1.4 Justificativa e relevância do tema.....……...……..……...…….…… 12
1.5 Objetivo Geral……...…… 13
1.6 Objetivos específicos……...… 13
1.7 Definição de termos……..…… 14
1.7.1 Jogos modo texto…… 14
1.7.2 Unicode……..……..…….…….…… 14
1.7.3 Emoji……..…… 14
1.7.4 Emoji Games..…..…...…....……15
1.7.5 Motor de Jogo (*Game Engine)*…… 15
1.7.6 Fonte…….……...…… 15
1.7.7 *Sprite* e *spritesheet*…….…….. 15
1.7.8 web game…..…….……..…….. 16
1.8 Materiais e Métodos……...…… 16
1.8.1 HTML5 canvas……..……...16
1.8.2 JavaScript……..……...17
1.8.3 Phaser….……...……..……17
1.8.4 Editor ou ambiente de programação (IDE)……..…… 17
1.8.5 *Debugger* do Chrome e Phaser……...…..…… 18
1.8.6 UML……...…... 18
1.8.7 Astah professional……..……18
1.8.8 Desenvolvimento de software Ágil……..…… 18
1.8.9 Engenharia de software para Games……..……19
1.8.10 Back-End……..…… 20
1.9 Limitações……..……20

1.10 Delimitações……..……..21
1.11 Suposições…… 21
1.12 Organização do estudo……..…… 21
2. ARCABOUÇO TEÓRICO(Revisão Bibliográfica)…… 22
3. METODOLOGIA DO ESTUDO…… 24
3.1 A equipe de Desenvolvimento……..…….. 24
3.2 Pré-produção……..…… 24
3.3 Pesquisa e desenvolvimento…….. 25
3.3.1 Seleção das ferramentas……..……... 25
3.4 O Projeto, especificação e plano de produção..…….. 26
3.5 Elicitação de Requisitos…….…….. 27
3.6 Desenvolvimento……..…… 32
3.6.1 Criação de ativos (*game assets)*……..….. 41
3.6.2 *Debugging*, *Tuning* e polimento……..…..41
3.6.3 Implementação……..……..….. 42
3.7 Pós-produção…….……42
4. RESULTADOS E IMPLICAÇÕES…… 42
5. SUMÁRIO, DISCUSSÃO E CONCLUSÕES…….…… 45
5.1 Discussão dos resultados……..…… 46
5.2 Implicações para a prática……..…… 46
5.3 Recomendações para pesquisas futuras……..…… 47
5.4 Sumário e considerações finais……..….. 48
6. CRONOGRAMA……..…… 49
7. REFERÊNCIAS BIBLIOGRÁFICAS……..…….. 49
Apêndice 1 – DOCUMENTO DE PROJETO DE JOGO (*Game Design Doc.)*…..55
Apêndice 2 – Plano de Negócio EMOJIGAME……..…..61
Apêndice 3 – Modelo Canvas para o EMOJIGAME……..…. 70
Anexo 1 – Edital de especificação de projeto do Lab. de Eng. de Software….71

1. INTRODUÇÃO

O tópico ou domínio do presente Trabalho de Conclusão de Curso é o projeto e desenvolvimento de Jogos Digitais com uso dos conhecimentos adquiridos no Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas (ADS).

O objetivo principal deste trabalho teve a intenção de ser uma contribuição no teste e demonstração da tecnologia que explora a possibilidade da utilização dos emoji e outros símbolos e caracteres da codificação unicode servindo como elementos gráficos (*graphic assets*), propondo uma nova categoria de jogos digitais baseados em texto ou em modo texto, para ser um facilitador no desenvolvimento de games e seu uso no ensino de programação.

O trabalho apresenta o relatório do projeto, desenvolvimento, implementação e apresentação do web game intitulado EMOJIGAME em dezembro de 2017 durante a disciplina de Laboratório de Engenharia de Software (IES301).

Neste também estão inclusos em apêndices esboços de documento de projeto de jogo (*Game Design Document*), plano de negócio e análise canvas como propostas para continuação da pesquisa e produção futura, propondo o desenvolvimento colaborativo gamificado contínuo, valendo-se dos Métodos Ágeis de Engenharia de Software, *frameworks* de desenvolvimento, gestão e governança provendo qualidade, transparência e automação dos processos administrativos do projeto.

1.1 Contextualização e Motivação

Jogos de Computador, ou Games, segundo Wang e Wu (Cooper e Scacchi, 2015, p. 32), estão sendo cada vez mais usados na educação para melhorar o desempenho acadêmico, a motivação e a dinâmica em sala de aula, não só para ensinar crianças em idade escolar, mas também para melhorar a qualidade da educação universitária em ciências da computação e engenharia de software.

Na disciplina de Linguagem de Programação (ILP010) fomos convidados pelo professor Doniseti a desenvolver um jogo simples de batalha naval utilizando a linguagem C, em modo texto, no prompt de comando. Desenvolvendo o exercício proposto, aprendemos como mover a “mira” - uma letra X - pelas coordenadas da tela, apagando e reescrevendo em posição adjacente, como a mudar as 16 opções

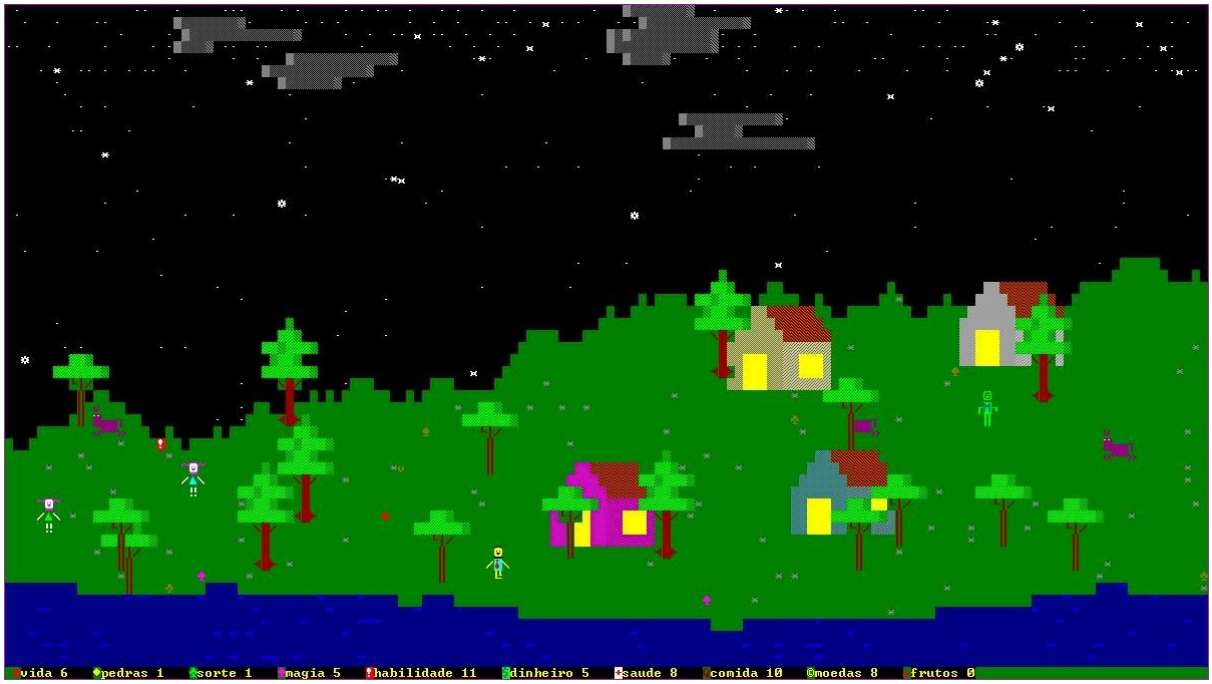
de cores de texto e fundo, produzir ruídos em diferentes frequências, e outras. A partir disso, adquiri confiança para tentar produzir um jogo, com a ideia de que se podemos mover uma letra também podemos compor um personagem ou veículo montado com símbolos e letras. Criei um “boneco de palito”, que chamo de ascii *stickman* (figura 1) para ser o personagem principal do jogo, junto com outros personagens não jogáveis (NPC’s). Também criei animais, casas, árvores, itens e obstáculos em um cenário gerado proceduralmente (figura 2) atualizando uma matriz com uma terceira dimensão de três camadas de valores. A primeira camada informava qual letra seria mostrada na tela. Na segunda, havia a informação do valor que representa as cores da fonte e do fundo e, na terceira, um atributo informando se o ponto era um obstáculo, objetos coletáveis, inimigo que causa dano, entre outros. Posteriormente, pesquisei e não encontrei outro jogo que já tivesse usado um personagem parecido. Na maioria dos jogos modo texto, usavam um único caractere ☺ (use alt + 1 para reproduzir o ASCII do smile) para representar o personagem, o que me animou a continuar com o projeto por, aparentemente, estar produzindo um jogo de certa forma inovador, mas as ideias que eu queria programar ficaram paradas na complexidade da programação em tempo real típica dos jogos de ação, devido ao processamento paralelo implementado com as *threads* em C, para que outras ações ocorressem ao mesmo tempo que as ações do jogador, entre outras dificuldades, até que eu me capacitasse em linguagens de programação mais modernas e complexas, que contam com bibliotecas, e engines para auxiliar o desenvolvimento de jogos, como o C++, C# e o Java.

Figura 1: ASCII stickman



Fonte: autor

Figura 2: cena capturada do jogo em C no prompt de comando



Fonte: autor

Posteriormente, já com conhecimentos e experiências em programação na interface gráfica, para web e dispositivos móveis, a evolução natural do projeto foi tentar migrar do limitado terminal ou prompt de comando, a codificação ASCII e as 16 opções de cores para refatorar com mais recursos, já imaginando como seria interessante se eu pudesse mudar a expressão do personagem com os *emoji smiles* e usar todas as outras figuras, símbolos e letras de outros sistemas de escrita.

Já com o tema do anteprojeto para meu Trabalho de Conclusão de Curso definido como “fazer jogos em modo texto com a codificação unicode”, eu pesquisava a programação de jogos para Windows com linguagem de programação C++, bibliotecas Allegro, SDL, DirectX, e outras tecnologias clássicas, sem conseguir progredir. Orientado por professores migrei para o desenvolvimento de *web games*, ainda sem ter muita consciência de todas as vantagens decorrentes dessa escolha. Imaginei no começo que seria apenas necessário programar o jogo para acontecer o desejado diretamente no hipertexto. Comecei a pesquisa com JavaScript sem usar nenhum *framework*. Aprendi a colocar texto com a codificação unicode no elemento canvas do HTML5, o que me permitiu facilmente ir muito além do que tinha imaginado ser possível fazer, chegando ao que chamo de *demoscene,* que nesse trabalho já representa o estudo inicial de viabilidade técnica do projeto do

uso de emoji para criação de jogos, pois possuía interatividade e mostrava bom desempenho e fluidez na movimentação dos elementos, sem as típicas “travadas” dos antigos web games feitos em flash.

No experimento demonstrado abaixo (figura 3), o personagem amarelo é controlado pelas setas direcionais do teclado enquanto o extraterrestre vai e volta pelo cenário, carregando frutas. A abelha fica girando e uma legenda passa pela cena.

Figura 3: web demoscene feita com JavaScript e HTML5 canvas



Fonte: autor

1.2 O caso (ou Projeto)

Durante a disciplina Laboratório de Engenharia de Software (IES301) formamos grupos e fomos “contratados” pelo prof. Lucas Baggio para especificar, projetar, documentar, implementar e apresentar para avaliação de professores e profissionais convidados, um software aplicando os conhecimentos em engenharia de software, programação e gerenciamento de projetos, conforme o documento reproduzido no anexo 1.

Este trabalho é um relatório do projeto experimental para produção do web game de ação em modo texto intitulado EMOJIGAME, realizado pela primeira turma

do curso de Análise e Desenvolvimento de Sistemas, do turno da manhã da Fatec Ribeirão Preto, que produziu e implementou um *web game* que testa a possibilidade tecnológica, que tentou responder a pergunta da pesquisa.

1.3 Formulação do Problema (ou Questão da Pesquisa)

A pesquisa busca resolver o problema da necessidade de desenhar, comprar ou contratar um artista para produção dos elementos gráficos (personagens, cenários, objetos, entre outros) que são usados no desenvolvimento de jogos digitais, fazendo da tecnologia proposta um facilitador para estudantes, testando a hipótese do aproveitamento dos emoji e outras figuras e caracteres da codificação unicode como ativos (*assets*) de arte gráfica em jogos casuais, reapresentando os antigos jogos baseados em texto em uma linguagem atual e com possibilidades infinitas de composição e demonstrando uma opção para a formação de desenvolvedores e programadores de jogos, respondendo à pergunta: É possível usar os emoji e outros caracteres e símbolos unicode como ativos gráficos para fazer jogos de ação modo texto?

1.4 Justificativa e relevância do tema

A temática dos emoji é um sucesso com centenas de jogos explorando as adoráveis figuras que já fazem parte da cultura popular mundial. É um universo em expansão com dezenas de novos emoji sendo lançados a cada ano. Mesmo assim, os *games* que são produzidos com o tema usam a tecnologia vigente de trabalhar com elementos *bitmap*, recriando artes dos emoji para serem usadas nos jogos, fazendo o jogo ficar maior e mais pesado e não se valendo da conveniência de poder mudar o tamanho, a cor, e aplicar efeitos em elementos de texto. Jogos “modo texto”, atualizados para o estado da tecnologia, facilitarão a internacionalização e localização (adequação legal e cultural de conteúdo), assim como a inserção de informação em tempo real e propaganda. O uso dos emoji *smiles* como sendo a cabeça do personagem representando as diversas expressões, das mãos emoji representando os gestos, o sapato e tênis representando o pé e etc na composição de personagens parece ser inédito e se bem desenvolvido e explorado, pode gerar Propriedade Intelectual.

O desenvolvimento de Jogos Digitais é um setor estratégico e apoiado pelo governo através de diversas iniciativas e editais de financiamento. A indústria de jogos movimenta mais de cem bilhões de dólares anuais, (Newzoo, 2016) equiparada à indústria farmacêutica, já ultrapassou a indústria do cinema e só perde para a indústria bélica e petrolífera. O Brasil ainda tem muito espaço para crescer neste mercado. O desenvolvimento de jogos na academia é divertido e enriquecedor e os conhecimentos e tecnologias adquiridos são aplicáveis além do domínio dos jogos. A capacitação na gestão de projetos de games, na governança em tecnologia da informação, empreendedorismo, sistemas de informação, e outros, aplicado à produção de jogos e integrado com a administração dos estudantes de Gestão de Negócios e Inovação pode ser reaproveitada na produção e gestão de outros jogos pela Empresa Jr. da Fatec, trazendo visibilidade e investimentos para nosso curso, e também assessorando desenvolvedores externos, fomentando o setor produtivo de desenvolvimento de jogos na nossa região.

1.5 Objetivo Geral

O objetivo da pesquisa foi dominar as tecnologias necessárias para testar a possibilidade do desenvolvimento de videogames de ação criados com letras e símbolos como opção tecnológica facilitadora para a criação de jogos, sem a necessidade de produzir e utilizar artes gráficas, para que estudantes de cursos regulares de programação possam ser atraídos para a produção de jogos com tecnologias populares e livres do pagamento de licenças. Também para que estudantes interessados em games se interessem por aprender programação, engenharia de software, banco de dados, gestão de projetos, governança, empreendedorismo, enfim, que vejam na formação em Tecnologia em Análise e Desenvolvimento de Sistemas um caminho para a capacitação em produção de jogos digitais.

O resultado esperado foi usar símbolos do conjunto unicode para produzir um web game que tenha um personagem no estilo “boneco de palito”, com face de emoji e corpo composto de caracteres interagindo com outros objetos, personagens e cenários totalmente compostos em modo texto, demonstrando a viabilidade da possibilidade tecnológica aventada.

1.6 Objetivos específicos

I. Testar a hipótese através do desenvolvimento de um game nos termos propostos.
II. Apresentar a inovação tecnológica como recurso didático facilitador para o desenvolvimento de videogames ser mais utilizado em cursos de programação.
III. Incentivar a retomada da produção de games baseados em texto.
IV. Propor a continuação do projeto de forma colaborativa.

1.7 Definição de termos

1.7.1 Jogos “modo texto” ou baseados em texto

Um jogo modo texto ou baseado em texto (ou ainda *ASCII game* e *ANSI game*) é um videogame que usa caracteres de texto em vez de trabalhar com gráficos em *bitmaps* ou vetores. Jogos modo texto são tipicamente fáceis de escrever e requerem menos poder de processamento. (TEXT-BASED, TEXT MODE, 2018) Normalmente é um termo usado para definir interfaces que só exibem fontes com caracteres monoespaçados e nessa obra o termo é reapropriado para designar a tecnologia proposta de trabalhar na interface gráfica só com elementos de texto.

1.7.2 Unicode

É a codificação universal de caracteres. O padrão que permite que computadores representem e manipulem, de forma consistente, texto em qualquer sistema de escrita existente (UNICODE, 2018) através de, atualmente, 137.439 caracteres, dentre eles, mais de mil emoji, peças do xadrez, damas, cartas de baralho e tarô, pedras do dominó, faces do dado, trigramas do i-ching, formas geométricas, símbolos, hieróglifos, e caracteres de praticamente todos os sistemas de escrita do mundo.

1.7.3 Emoji

(絵文字, *lit.* ***pictograma***) é uma palavra derivada da junção dos seguintes termos em japonês: e (絵, "imagem") + moji (文字, "letra"). Com origem no Japão, os

emoji são o conjunto de mais de mil pictogramas usados em mensagens eletrônicas e páginas web.

1.7.4 Emoji Games

Já existem centenas de jogos com a temática dos Eemoji, sendo eletrônicos, baralhos, jogos de tabuleiro e outros, mas mesmo quando se propõe a fazer um jogo feito inteiramente com emoji, preparam imagens bitmap dos emoji ou até modelam bonecos de emoji em 3D em vez de buscarem as imagens já disponíveis no sistema operacional.

Outro aspecto que nosso projeto difere de outros emoji games é que em todos os casos o personagem do jogo é o emoji em si (como aparecem em “Emoji, the movie”). A composição de um personagem com vários emoji, com os smiles sendo a cabeça com várias expressões e as diversas mãos formando gestos, a camisa sendo o corpo e o sapato ou tênis os pés do personagem, parece ser inédita.

1.7.5 Motor de Jogo (*Game Engine*)

Game engine é um ambiente de desenvolvimento de software (API) ou conjunto de bibliotecas que auxiliam o desenvolvimento de jogos. Apresenta funcionalidades que incluem: Um motor gráfico para a renderização de gráficos 2D ou 3D; um motor de física que faz simulações e detecta colisões; suporte a animação; sons; inteligência artificial; *networking*; gerência de memória; gerência de arquivos; gerência de linha de execução; e suporte a uma linguagem de script. (GAME ENGINE, 2018).

1.7.6 Fonte

Uma fonte tipográfica é um padrão, variedade ou coleção de caracteres tipográficos com o mesmo estilo, atributos e tamanho. Na programação web podemos utilizar as fontes *default*, disponíveis no sistema operacional ou indicar no código um local de origem na web de onde a fonte é carregada para ser usada. (FONTE TIPOGRÁFICA, 2018)

1.7.7 *Sprite* e *spritesheet*

Spuy (2012, p. 374) define que sprites são objetos de jogos animados que se movem pela tela. Eles podem ser personagens do jogo, itens que os jogadores podem coletar, projéteis, partículas em uma explosão, ou mesmo grandes fundos. A tecnologia que propomos nesse projeto é toda baseada em elementos de texto e dispensa o uso de s*prites*.

1.7.8 web game

Browser game ou jogo on-line é um jogo que se utiliza de navegadores e da internet. Possui várias vantagens em relação a outras arquiteturas, pois não exige instalação. (BROWSER GAME, 2018) podem ser executados em vários sistemas operacionais, consoles, mobile, tablets e outros dispositivos. A programação de web games “pode ser mais fácil. A programação de jogos é bem conhecida por ser uma das formas mais difíceis de programação. Isto é especialmente verdade quando você usa um das linguagens de programação tradicionais, como C++. “ (Harris, 2013, p. 10). Web games facilitam o desenvolvimento de jogos porque vários aspectos da programação são executados e gerenciados pelo *browser*. Dispensam mídia física, publicadoras e lojas de aplicativos, podendo ser apresentados diretamente ao público e facilmente são incorporados às redes sociais.

1.8 Materiais e Métodos

Nessa sessão, adaptando a metodologia ao desenvolvimento de jogos, apresentamos a arquitetura e tecnologias utilizadas no projeto experimental relatado como os materiais da pesquisa e o modelo de desenvolvimento ágil como o método aplicado.

1.8.1 HTML5 canvas

O elemento canvas do HTML5, ensina Rowel (2011, p.16), é uma tag que podemos inserir num documento HTML que define uma área no navegador usada para desenhar gráficos, em tempo real, com a linguagem JavaScript. Como o

canvas é um bitmap, cada pixel desenhado no canvas substitui pixels que estavam no lugar. Segundo Fulton (2013, p.23) o canvas trabalha em modo imediato, no qual o HTML5 redesenha completamente a tela de bitmaps a cada *frame* através do chamado da API Canvas pelo JavaScript. O canvas possui vários métodos para desenhar linhas, caixas, círculos, texto e adicionar imagens. (w3schools, 2018).

1.8.2 JavaScript

JavaScript ou ECMAScript é uma linguagem de programação interpretada, principal linguagem para a programação *client-side* em navegadores web. É a linguagem de programação mais utilizada do mundo (JavaScript, 2018). Spuy (2012) apresenta a linguagem dizendo que o JavaScript está no coração de um jogo HTML5, que quase tudo isso é importante no web game será controlado pelo JavaScript de alguma forma e que JavaScript é um dos mais fáceis linguagens de programação para aprender. JavaScrpt possui várias ferramentas de apoio como o JSLInt (https://www.jslint.com/) serve para conferir e otimizar o código, ferramentas de minificação que comprimem o código, ferramentas de obsfucação que dificultam a leitura do código fonte, e dezenas de frameworks.

1.8.3 Phaser

Phaser é um framework *open source* para a produção de web games para computadores, mobile e consoles. Está disponível para download grátis, desenvolvido pela produtora de games PhotonStorm. Não há cobrança de taxas mesmo desenvolvendo projetos comerciais. Possui mais de 700 exemplos de código para estudo, uma comunidade ativa e centenas de jogos publicados. Está na versão 3.1, totalmente reformulada.

1.8.4 Editor ou ambiente de programação (IDE)

Qualquer editor de texto é o suficiente para programar em JavaScript. No desenvolvimento do EMOJIGAME foram testados diversas IDEs, como o Visual Studio, Brackets, Eclipse e Visual Studio Code, todos com boa performance e a escolha não interfere nos resultados obtidos.

1.8.5 *Debugger* do Chrome e Phaser

O desenvolvimento com JavaScript é testado e depurado no Browser. Quando algum código não funciona o jogo simplesmente não roda e precisamos apertar o F12 para consultar as mensagens de erro.

> O depurador de JavaScript que faz parte do console do seu navegador também tentará dizer a você o que acha que está errado com o seu programa. Mas, como tenho certeza que você já percebeu, as mensagens de erro são extremamente enigmáticas e, mesmo Se você conseguir decifrá-los, eles podem não dizer o que está errado. Spuy (2012, p. 109).

> Phaser vem embutido com uma instância de uma classe de depuração anexada a cada jogo. (...) O objeto de depuração funciona desenhando várias informações, formas, etc. em uma tela. Os recursos de depuração devem ser usados somente para depuração e desativados quando você envia a versão real do jogo. Ao desenvolver aplicativos JavaScript no navegador, você pode estar acostumado a usar a saída do console do desenvolvedor para obter informações de depuração. O problema de usar isso para jogos é que as informações com as quais mais nos preocupamos são atualizadas em todos os quadros, em vez de se basearem em eventos. Se saíssemos da posição de *sprite*, por exemplo, ele rolaria continuamente o console e não poderíamos rastrear as coisas com muita facilidade. É por isso que a capacidade de renderizar informações de depuração para a tela do nosso jogo é tão importante. (HOW TO…, 2018)

1.8.6 UML

A UML**,** Linguagem de Modelagem Unificada é uma linguagem-padrão para a elaboração da estrutura de projetos de software. Na engenharia de software, a UML é uma linguagem de modelagem que permite representar um sistema de forma padronizada (com intuito de facilitar a compreensão e pré Implementação). (UML, 2018)

1.8.7 Astah professional

Astah é o software que foi usado para elaboração dos diagramas UML do projeto. É gratuito para estudantes cadastrados.

1.8.8 Desenvolvimento de software Ágil

O desenvolvimento de jogos digitais é exemplo típico onde métodos ágeis são a melhor escolha para modelo de desenvolvimento, como Sommerville (2011) indica, games deveriam sempre ser desenvolvidos usando uma série de protótipos. É também chamado modelo de protótipo evolutivo.

Keith (2010, p. 24) mostra como os valores do manifesto ágil são aplicados ao desenvolvimento de Games:

1. Os indivíduos e suas interações acima de procedimentos e ferramentas. O desenvolvimento de games requer a interação entre profissionais de diversas áreas que precisam trabalhar bem em equipe.

2. O funcionamento do software acima de documentação abrangente. O jogo funcionando é mais importante que um projeto abrangente.

3. A colaboração com o cliente acima da negociação e contrato. A colaboração entre o desenvolvedor e o editor é o mais importante. O cliente tem que confiar que o desenvolvedor está gastando dinheiro com sabedoria. O desenvolvedor tem que confiar que o cliente não cancelará o andamento contrato sem uma boa razão.

4. A capacidade de resposta a mudanças acima de um plano pré estabelecido. A abordagem ágil é planejar o que é conhecido e iterar contra o que não é conhecido. Criar um jogo divertido e inovador é sempre desafiador.

> Os 12 princípios do desenvolvimento ágil são os seguintes:
> Garantir a satisfação do cliente, entregando rápida e continuamente software funcionais;
> Software funcionais são entregues frequentemente (semanal, ao invés de mensal);
> Software funcionais são a principal medida de progresso do projeto;
> Até mesmo mudanças tardias de escopo no projeto são bem-vindas.
> Cooperação constante entre as pessoas que entendem do 'negócio' e os desenvolvedores;
> Projetos surgem por meio de indivíduos motivados, devendo existir uma relação de confiança.
> Design do software deve prezar pela excelência técnica;
> Simplicidade;
> Rápida adaptação às mudanças;
> Indivíduos e interações mais do que processos e ferramentas;
> *Software* funcional mais do que documentação extensa;
> Colaboração com clientes mais do que negociação de contratos;
> Responder a mudanças mais do que seguir um plano. (MANIFESTO ÁGIL, 2018)

1.8.9 Engenharia de software para Games

Cooper e Scacchi (2015) explicam que Jogos de computador são ricos, complexos e às vezes enormes softwares aplicativos. São significantes, interessantes e muitas vezes envolventes aplicações para a pesquisa inovadora de técnicas e tecnologias em engenharia de software. O surgimento da indústria de games, com grandes estúdios criando jogos para o mercado global, logo deixou clara a necessidade de adotar técnicas e práticas modernas de engenharia de software, do contrário, provavelmente os desenvolvedores sofreriam problemas em manter ou expandir os games, que é o destino comum de sistemas cuja complexidade não reconhecida cresce além das habilidades de programação convencionais de seus desenvolvedores.

Os diferenciais da engenharia de software para games estão na elicitação de requisitos, que são focados na qualidade do jogo e interesse do público. A metodologia de desenvolvimento é a do protótipo evolutivo, sempre com a utilização de diversos métodos ágeis aplicáveis.

1.8.10 Back-End

Davey (2018) explica que, para desenvolver jogos com o Phaser, é necessária a instalação de um servidor web local. Por questões de segurança não é possível rodar um código JavaScript com todos os recursos habilitados localmente. Em nosso experimento usamos o servidor Glassfish e Java, com JSF com Primefaces para cadastro, login e acesso ao banco de dados PostgreSQL do placar de pontuações.

1.9 Limitações

O protótipo do jogo foi desenvolvido primeiramente para computadores com Windows 10 e navegador Chrome.

Nesta pesquisa não utilizamos a instalação de fontes, então os resultados são dependentes do Sistema Operacional do usuário.

O projeto esteve contido ao cronograma e tivemos apenas dois meses para desenvolver o software e produzir a documentação. O produto apresentado

tecnicamente pode ser dito como um jogo completo nos requisitos apresentados, mas também representa um protótipo a ser lapidado e incrementado, além de o primeiro ciclo do desenvolvimento de um produto comercial completo.

1.10 Delimitações

Optamos pelo desenvolvimento de web game, mas não descartamos a possibilidade de futuramente portar a tecnologia para outras arquiteturas, como o android e IOS nativos, usar o Unity ou C++, entre outras.

Evitamos a utilização de qualquer tecnologia proprietária que exigisse o pagamento de licença, assim como não usamos de material com direitos autorais.

Nesta primeira versão não foram consideradas as possibilidades de formatação, efeitos e animações CSS. Também não foram explorados todos os recursos gráficos do HTML5 canvas, como as operações de composição.

1.11 Suposições

Para consideração dos resultados desse estudo, a banca de convidados que avaliou a apresentação do projeto representou possíveis investidores e os comentários dos alunos, amigos e internautas que testaram o jogo fazem o papel do retorno (*feedback*) dos jogadores.

1.12 Organização do estudo

Este trabalho é apresentado em cinco partes:

Parte 1: é a apresentação e inclui introdução, contextualização, definição do problema, objetivos, definição de termos, materiais e métodos, limitações, suposições e organização do estudo.

Parte 2. Revisão Bibliográfica, que inclui a descrição do material de estudo suficiente para produzir web games.

Parte 3: Metodologia do Estudo. Descreve a metodologia usada nesse estudo.

Parte 4: Resultados e implicações. Apresenta os resultados obtidos no desenvolvimento do game.

Parte 5: Sumário, discussão e conclusões, implicações para a teoria e prática e faz recomendações para pesquisas futuras.

2. ARCABOUÇO TEÓRICO (Revisão Bibliográfica)

Muitos livros podem ser encontrados na Internet tratando sobre o projeto e desenvolvimento de jogos. Foram selecionados aqui os mais atuais e que tratam do projeto de jogos com as tecnologias usadas no desenvolvimento do *web game*.

A engenharia de software geral é bem estudada por Sommerville (2011) e Pressman (2011). Rucker (2002) e Flynt (2005) tratam da engenharia de software para desenvolvedores de jogos. Cooper (2014) compilou vários estudos da engenharia de software para games; Keith (2010) explicou o desenvolvimento ágil de jogos com scrum; Irish (2005), Uippa e Borst (2010) trataram da produção de jogos. Bethke (2004), Rollings (2004), Cohen e Bustamante (2012) e Novak (2012) trataram do desenvolvimento e produção de jogos; Walker (2003) e Bates (2015) Escreveram guias de mercado para desenvolvedores de jogos. Steiinberg (2007) trataram do Marketing para Videogames. Também convém uma pesquisa no Global Game Market Report, disponível em https://newzoo.com/insights/articles/ que traz os números do mercado mundial de games. Outra fonte de formação valiosa é o curso online gratuito oferecido pelo Ministério da Cultura e Abragames sobre o mercado de games, dividido em três módulos disponíveis em na plataforma lúmuna: “panorama, carreira e oportunidades”; “dicas e desafios para empreendedores”; e “Internacionalização no setor de games”. Disponíveis em https://lumina.ufrgs.br/course/view.php?id=42 (id = 48 e 49 para acessar os outros módulos do curso).

Para uma formação no projeto (*design*) de jogos podemos consultar vários livros que tratam da história dos games, que discorrem sobre os diferentes tipos de jogos, os vários tipos de jogadores e motivos por que jogam, e os elementos que tornam um jogo interessante. Nesse sentido Rouse (2001), Pedersen (2003), Fullerton (2004), Bates (2004), Oxland (2004), Perry (2009), Rogers (2010), Moore (2011) e Sylvester (2013), Schell (2015) e Mayer (2016) apresentaram o geral de game design; Crawford (1997/2003) e Flanagan (2009) descreveram sobre a arte do projeto de jogos; Koster (2013) teorizou sobre o que torna um jogo divertido; Despain (2013) elenca 100 princípios para o *Game Design;* Sheldon (2004) e

Isbister (2006) trabalharam com o design de personagens para jogos; Fox (2003) tratou do projeto de interface para jogos. Borries, Walz e Böttger (2007) organizaram textos que analisam os espaços, arquitetura, ambientação e mapa de fases dos mais variados tipos de jogos; Feil e Scttergood (2005) se dedicaram ao design de fases; Swink (2009) traçou um guia de sensações virtuais para o game designer. Buck (2015) ensinou algoritmos para a construção de labirintos; Jull(2010) e Trefay(2010) discorreram sobre a retomada da produção de jogos casuais. Adams (2012) trouxe o projeto avançado com a mecânica dos jogos. Também é um recurso valioso acompanhar os cursos do MIT Open Courseware, especialmente o *Creating Video Games,* do professor Tam e outros (2014).

No domínio da programação de web games e também saber o básico de JavaScript o ponto de partida para o estudante é o Mozila Development Network que traz um panorama de todas tecnologias envolvidas. (https://developer.mozilla.org/en-US/docs/Games) e o tutorial passo a passo do w3schools (https://www.w3schools.com/graphics/game_intro.asp) que traz o básico para o desenvolvimento de web game simples. Para estudos mais aprofundados temos vários títulos, como Gold (2004) que tratou do desenvolvimento de jogos orientado a objetos, Rowell (2011) e Fulton (2013) apresentaram receitas do HTML5 canvas. Geary (2012), Spuy (2012), Williams (2012), Shankar (2012), Cássio (?) e Egges (2014) ensinaram os primeiros passos em programação de jogos HTML5; Bunyan (2015) ensinou HTML5 com CSS e JavaScript; Kurianovich (2012) e McAnlis (2014) Trouxeram dicas profissionais para o desenvolvimento de jogos HTML5. Rettig (2012) tratou do desenvolvimento de jogos HTML5 para dispositivos mobile. Dyrr (2016) e Faas (2017) editaram manuais para criação de jogos com o Phaser. Bibat (2018) escreveu um workshop online que ensina a fazer um clone do game 1942 usando o Phaser.

Palmer (2005), Bourgh e Bywalec (2013) escreveram sobre a física para programadores de jogos, Dobre e Ramtal (2014) explicaram como incorporar física na programação JavaScript para jogos, animação e simulação. Watson (2018) criou o gamemechanicexplorer.com, “Uma coleção de exemplos concretos para várias mecânicas de jogo, algoritmos e efeitos. Os exemplos são todos implementados em JavaScript usando a estrutura do jogo Phaser”.

Silveira (2015) tratou da criação de jogos *multiplayer* com HTML5.

Smus (2013) ensinou como usar a *web audio API*, Childs (2007), Collins (2008) e Marks (2009) trataram da criação de música e sons para games.

Meyer (2010) usou jogos para ensinar HTML5 e JavaScript. O'neil e Perez (2008) organizaram estudos sobre o uso de videogames para o ensino de adultos.

3. METODOLOGIA DO ESTUDO

Este trabalho é o relatório de uma pesquisa experimental exploratória aplicada, realizada com método hipotético-dedutivo, avaliando os resultados de forma qualitativa, as tecnologias necessárias para produzir jogos digitais nos moldes propostos, descrevendo o protótipo desenvolvido como proposta de solução para o problema apresentado.

3.1 A equipe de Desenvolvimento

O projeto foi realizado pela primeira turma do curso de ADS período manhã, apresentada em ordem alfabética, com os cargos que exerceram no projeto, doravante denominados "Equipe de desenvolvimento":

Eurico Ferreira de Souza Jr.
Programador do Jogo / Gerente do Projeto
Jessica Silva de Oliveira
Engenharia de Software
Pedro César de Freitas Penarbel
Desenvolvedor *Back-end*
Vinícius Alves Silva
Engenharia de Software
Welson Rodrigo Gomes de Oliveira
Desenvolvedor *Back-end*

3.2 Pré-produção

O primeiro desafio da equipe, na fase de concepção do projeto, foi escolher que tipo de software e qual problema este poderia resolver. Perdemos bastante tempo disponível para a realização do trabalho debatendo entre as possibilidades de produzirmos software comparativo de preços, aplicativo para pedir a entrega de comida pelo celular, ponto de venda, web TV, gerenciador pessoal de saúde, ERP, entre outros. Eu já tinha essa ideia definida para meu TCC e a *demoscene* (figura 3) feita só em JavaScript. O voto de minerva e o argumento que venceu vieram do

aluno Paulo Mello, de outra turma, que cursava algumas matérias em comum e conhecia o que eu estava desenvolvendo quando disse com injustificável otimismo “Façam o jogo, já está pronto!”. O entusiasmo contagiou a equipe em termos já que o desenvolvimento de jogos não faz parte da formação em ADS, mas aceitamos o desafio e ficou combinado que eles fariam a engenharia de software, documentação, qualidade e o *back-end* com o projeto e programação do banco de dados enquanto eu transformaria a *demoscene* em um videogame.

3.3 Pesquisa e desenvolvimento

Programar jogos é divertido e um bom caminho para a capacitação em várias tecnologias. Qualquer conhecimento em JavaScript é precioso e útil em vários outros domínios da programação.

3.3.1 Seleção das ferramentas

Como focamos a pesquisa em aprender a programar fazendo jogos para um dia ensinar programação ensinando a fazer jogos, naturalmente desprezamos qualquer software que se propõe a fazer jogos sem a necessidade de programação ou escondem o código do usuário, assim como evitamos trabalhar com *engines* sofisticadas com longa curva de aprendizagem, programação com linguagens sofisticadas como C#, C++, poderosas, mas desafiadoras para novatos, o uso de tecnologia proprietária, e como a ideia é fazer jogos modo texto foi deixado de lado tecnologias que trabalham com modelos 3D. Cogitamos usar o Unity, mas a possibilidade foi preterida ao constatarmos que o esquema para produzir jogos 2D em modo texto envolveria criar objetos 3D sem profundidade com o texto anexado, que ficariam na mesma linha e o movimento seria acompanhado pela câmera num ambiente 3D simulando 2D. Além disso, trabalhar com os emoji e unicode e ter a liberdade de exibir texto com efeitos já alcançados no canvas do HTML5, pelo menos na época que a pesquisa foi realizada, já ia além da versão gratuita do Unity, exigindo a compra de extensão (*plugin)*.

Primeiramente com o objetivo de agilizar o desenvolvimento, optamos por utilizar uma *engine* ou *framework* para web games. O Professor Lucas Baggio explicou que não é um demérito para o projeto e que deveríamos utilizar como

critério de seleção da ferramenta a quantidade de projetos desenvolvidos e se existe uma comunidade ativa da tecnologia. Segundo Cooper e Scacchio (2015, p.271) a seleção de um ambiente de desenvolvimento de jogos restringe ou predetermina que tipo de jogo de computador pode ser mais prontamente desenvolvido. Existem dezenas de *engines*, como podemos ver em https://html5gameengine.com/ e http://www.gamepix.com/blog/the-big-list-of-2d-html5-games-engines/.

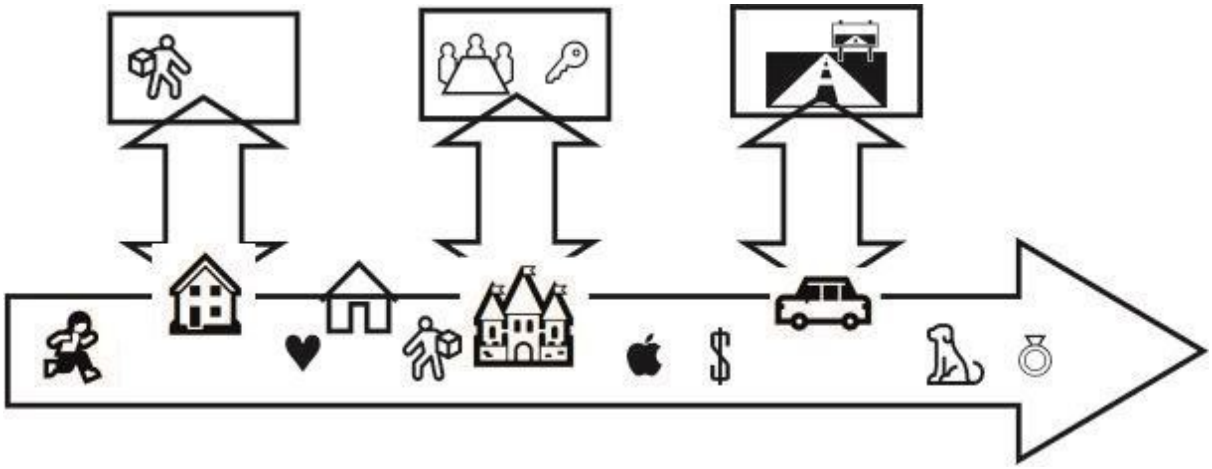
O site Our Code World (Best…, 2018) publicou um artigo que coloca o Phaser como melhor opção. Phaser possui mais de 19 mil repositórios no GitHub e uma comunidade ativa em http://www.html5gamedevs.com/.

Estudar, reproduzir e modificar os mais de 700 exemplos de código disponíveis no site do Phaser foi uma boa estratégia para adquirir familiaridade com o *framework*. As tecnologias tiveram que ser adaptadas, porque o processo normal da geração e animação das imagens no de desenvolvimento de games é baseada na preparação e uso de *sprites*.

3.4 Projeto, especificação e plano de produção

O objetivo era fazer um jogo de ação estilo *side scrolling runner*, ou jogo com visão lateral onde o personagem principal só pode ir para os lados (SIDE…, 2018), com as ações típicas de recolher itens e evitar obstáculos e inimigos. Posteriormente seriam adicionadas novas fases ao percurso principal para testarmos novos *gameplays*, conforme a arte conceitual abaixo:

Figura 4: arte conceitual da estrutura das fases no percurso principal.



Fonte: autor.

Foram empregados métodos ágeis de engenharia de Software aplicados ao desenvolvimento de jogos, mas o contrato (anexo 3) pedia documentação de especificação de requisitos entregue antes do desenvolvimento do software. Criamos maquetes das telas do jogo (figura 5), diagramas UML, MER e texto definindo a natureza, escopo, complexidade do problema e requisitos do projeto.

A estratégia foi cumprir os requisitos mínimos necessários e ter um software apresentável o mais rápido possível e posteriormente ir acrescentando melhorias.

Figura5: maquetes das telas dos estados do jogo.



Fonte: autor.

3.5 Elicitação de Requisitos

A primeira questão foi elicitar quais seriam os requisitos mínimos para um videogame ser considerado completo. Elencamos requisitos mínimos como ser divertido, possuir sons, uma mecânica consistente, condições de ganhar e perder,

tela de abertura, tela de fim de jogo, capacidade de salvar o resultado do jogo e apresentar num ranking. Também elencamos requisitos desejáveis como fazer o jogo funcionar bem em mobile, a possibilidade de cadastro pelas redes sociais, jogo *multiplayer*, inserir conteúdo relacionado à Fatec e aos conteúdos das matérias, entre outras ideias. Segundo Gose (2018) um protótipo é um jogo operacional que aceita *imputs*, move componentes do jogo, anima e reage aos objetos internos do jogo.

Cooper e Scacchi (2015, p. 269) alertam que desenvolvedores de jogos aprenderam que apenas fornecer um jogo cujo software funcione como pretendido (isto é, atende aos seus requisitos funcionais) é muito menos relevante do que fornecer um jogo que agrade diversos grupos de jogadores. Por isso não é surpreendente ver a indústria mudando seu foco para jogos que podem ser incrementalmente desenvolvidos e lançados com um conjunto mínimo de recursos de jogo e o sucesso inicial de um novo jogo (por exemplo, análises de jogadores online, vídeos de jogo criados pelo usuário e taxas de retorno de micro transações) determinará se mais recursos serão adicionados ou integrados, percebendo que o sistema de jogos está sendo desenvolvido como um serviço de jogo interativo online, e não apenas como um produto de software.

A seguir, reproduzimos parte da documentação de especificação de requisitos produzida pela equipe de desenvolvimento e entregue no início do projeto.

Quadro 1: Requisitos Funcionais.

ID	Descrição	Depende de
RF01	O personagem do jogador deve ter ao menos as ações de andar/correr, pular, abaixar, coletar itens, interagir com os personagens do jogo e objetos do cenário.	
RF02	O jogo deve ter música e sons para as ações.	RN2
RF03	O jogo deve ter um sistema de pontuação para itens coletados, missões concluídas, e outros eventos.	
RF04	O jogador deve ter um inventário de itens coletados e missões realizadas.	
RF05	O sistema deve ter a opção de deixar o usuário jogar sem estar logado (Quick play)	-
RF06	O sistema deve ser capaz de cadastrar os jogadores	RN06
RF07	O sistema deve mostrar o ranking dos jogadores	RF04
RF08	O sistema deve dar a opção do jogador salvar o jogo.	RN06
RF09	O sistema deve registrar as evoluções e conquistas dos jogadores	RN06
RF10	O sistema deve ser capaz de disponibilizar novas fases	RN05

	conforme o jogador progride.	
RF11	O sistema deve ter tela de abertura (Splash screen)	
RF12	O sistema deve mostrar os créditos da produção no final das partidas.	

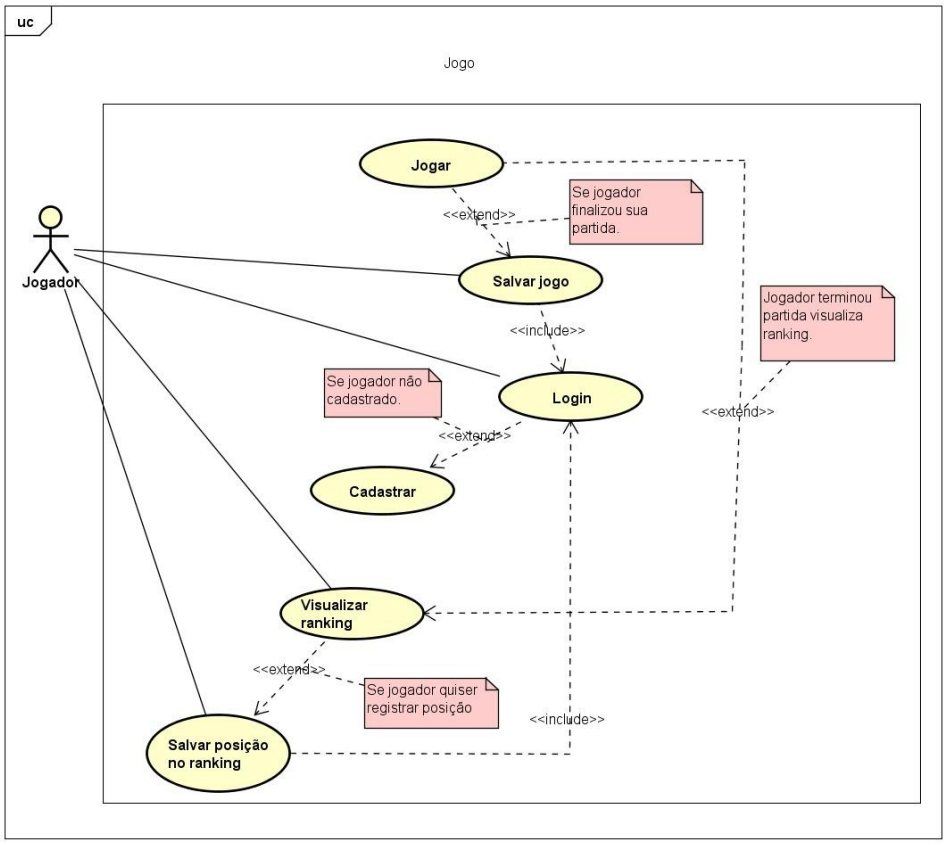
Fonte: autor

Quadro 2: Requisitos não funcionais.

ID	Descrição	Depend e de
RN01	Os gráficos do jogo devem ser compostos por EMOJI, outras letras e símbolos do UNICODE, e seguir essa identidade visual.	
RN02	O jogo deve sincronizar ações, sons e movimentos com Web Workers (threads JavaScript).	
RN03	O jogo deve ter ação rápida e fluida.	
RN04	O web game deve ser projetado com a arquitetura MVC.	-
RN05	O servidor do web game deve ser desenvolvido com a tecnologia Java EE.	
RN06	O sistema deve possuir Banco de Dados normalizado	RN05
RN07	O sistema deve se possível permitir logar pelas redes sociais	RF06

Fonte: autor

Figura 6: diagrama de casos de uso



Fonte: autor.

Descrição dos Casos de Uso

Ator

- Jogar: A função desse ator é jogar o jogo, fazer o login, cadastrar, salvar o jogo, visualizar o ranking.

Casos de Uso:

- Jogar: Esse caso de uso refere-se ao ato do jogador jogar o seu jogo, observe que contém um caso de uso **salvar jogo** onde é opcional, ficando a critério do jogador salvar ou não seu jogo, como também contém o caso de uso **visualizar ranking** também opcional, pois ao jogar o jogador pode visualizar o ranking.
- Salvar jogo: Esse caso de uso refere-se ao jogador ter a possibilidade de salvar o jogo, observe que contém um caso de uso **login** sendo obrigatório que o jogador efetue o login para salvar o jogo.
- Login: Esse caso de uso refere-se ao jogador logar no jogo, observe-se que contém um caso de uso **cadastrar**, sendo opcional pois caso o jogador não for cadastrado ele pode efetuar o cadastrado.
- Cadastrar: Esse caso de uso refere-se ao jogador fazer o cadastro no jogo.
- Visualizar ranking: Esse caso de uso refere-se ao jogador visualizar sua posição e dos demais jogadores no ranking do jogo, observe-se que contém um caso de uso **salvar posição no ranking**, pois caso o jogador finalizou sua partida no jogo ele pode registrar sua posição, deixando salvo.
- Salvar posição no ranking: Esse caso de uso refere-se ao jogar salvar sua posição no ranking, observe-se que contém um caso de uso **login** de obrigatoriedade, pois para que sua posição seja salvo é preciso que o jogador esteja logado no jogo. (Equipe de desenvolvimento)

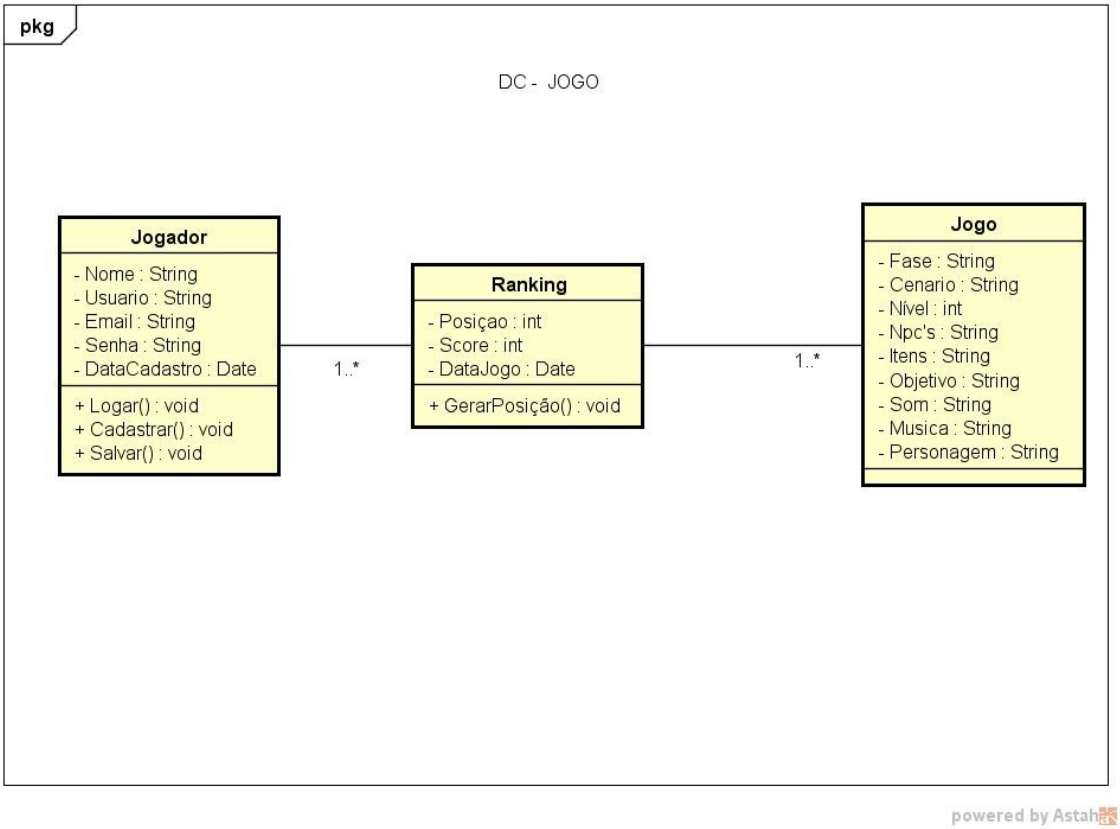
Quadro 3: Documentação do caso de uso: Login

Nome do Caso de Uso	**Login**
Ator Principal	Jogador
Ator Secundários	Servidor
Resumo	Esse caso de uso descreve as etapas onde o jogador faz o login para acessar o jogo.
Pré-Condições	Para fazer o login no jogo é preciso que o jogador já esteja cadastrado, pois caso não esteja cadastrado não é possível fazer o login.
Pós-Condições	
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1. Acessar o site do jogo.	2. Solicitar login e senha.
3. Informar login e senha.	4. Permissão válida.
5. Selecionar opções, como jogar, visualizar ranking.	6. Mostrar opção solicitada pelo jogador.
Fluxo de Exceção	
Ações do Ator	**Ações do Sistema**
	1. Caso login/senha esteja incorreto, emitir mensagem de erro

Restrições/Validações	Validar login/senha, mensagem de aviso validando.

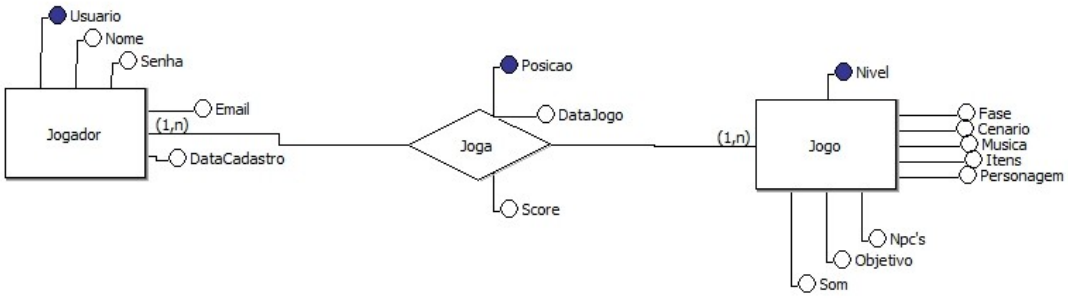
Fonte: autor

Figura 7: diagrama de classes



Fonte: autor

Figura 8: Modelo Entidade Relacionamento



Fonte: autor

3.6 Desenvolvimento

Os métodos mais importantes para desenvolvimento de *web games* em modo texto no HTML5 canvas são: CanvasRenderingContext2D.filltext(“text”, x, y), que desenha o texto nas coordenadas dadas (trocando o filltex por stroketext desenha o contorno do texto); CanvasRenderingContext2D.textAlign, que faz o ajuste de alinhamento de texto. Possíveis valores: start (padrão), end, left, right or center; CanvasRenderingContext2D.textBaseline que configura oalinhamento da linha de base. Valores possíveis: top, hanging, middle, alphabetic(padrão), ideographic, bottom; CanvasRenderingContext2D.direction, que controla a direcionalidade (em que sentido o texto é escrito). Valores possíveis: ltr, rtl, inherit(herdado, que é o padrão). O elemento texto aceita os métodos de transformação do contexto2D do canvas, como p.ex. Rotate(), Scale(),Translate(), Transform(), entre outros. O Phaser possui seus métodos próprios para inserir texto com text.game.add.text(x,y,”text”) e alinhar texto com text.align. O parâmetro anchor define o ponto de origem da figura, por onde ela vai ser fixada e girar. O padrão (0, 0) é o canto superior esquerdo. (0,5, 0,5) é o centro. (1, 1) é o canto inferior direito. O parâmetro angle pode girar o elemento, tendo o anchor como eixo. Ao adicionarmos um elemento de texto, devemos observar três parâmetros: em que coordenada estamos colocando o texto, por onde o texto estará “ancorado” e qual alinhamento de texto se está usando.

Faas (2017, p. 56) explica que a primeira coisa que devemos entender sobre um motor de jogo é o loop de jogo. Para um jogador perceber o movimento em um jogo suave e ininterrupto, o jogo precisa executar algum código idealmente 30 vezes por segundo, embora possa cair para cerca de 20 e ainda ser aceitável. O método de entrada irá receber o que o usuário está fazendo atualmente com seus dispositivos de interface, principalmente mouses e teclados, atualizando estado do jogo é onde toda a jogabilidade é implementada. Imagens são movidas, o áudio é acionado para tocar e as colisões são processadas no método de atualização. Finalmente, o método de renderização limpa a tela inteira e redesenha cada elemento visível no jogo.

A programação de um web game começa pela criação de um diretório (pasta) com o nome do projeto, onde os arquivos são salvos, organizados em sub-diretórios para scripts, assets, entre outros.

A execução do jogo começa por um arquivo index.html que declara o canvas onde as imagens serão apresentadas e os scripts que serão executados para promover toda a ação. O framework Phaser é um script que pode ser baixado em duas versões: phaser.js e phaser.min.js, o primeiro é usado durante o desenvolvimento para entendermos as mensagens do debugador e posteriormente é substituído pela segunda versão, minificada, para diminuir o download e melhorar a performance na versão de produção.

Abaixo, reprodução de parte do arquivo index.html onde são informados os scripts usados pelo web game:

```
<!DOCTYPE html>
<html>
   <head>
MI    <title>EM JI GAME</title>
      <meta name="viewport" content="width=device-width">
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <script src="scripts/Boot.js"></script>
      <script src="scripts/Preloader.js"></script>
      <script src="scripts/Game.js"></script>
     <script src="scripts/phaser.min.js" type="text/javascript"></script>
      <link href="styles/login.css" rel="stylesheet" type="text/css"/>
    <script type="text/javascript"
src="http://ajax.googleapis.com/ajax/libs/jquery/1.3.1/jquery.min.js"></script>
      <script type="text/javascript">
         $(document).ready(function () {
```

(...)

Omitimos a parte do código que salva o ranking no banco de dados por ir além do escopo desse trabalho, mas todo o código pode ser consultado no site do jogo publicado, simplesmente pedindo para exibir o código fonte (tecla f12). Em seguida, no mesmo arquivo index, um *script* inicia o jogo declarando um novo jogo no Phaser com a dimensão de 1280 x 720 pixels (720p, 2018), bem acima do proposto na maioria dos exemplos e tutoriais para web games, para testar a performance e buscar compatibilidade com outras mídias e estar adequado ao padrão de HDTV.

“Jogos Phaser são organizados em torno de estados. Pense nos estados do Phaser como as diferentes partes do jogo” (MAKING…, 2018)

> Os estados Phaser são uma implementação de uma estrutura de programação chamado de "máquina de estado finito" (FSM). Devido à sua natureza como FSMs, Phaser irá executar os métodos dentro do estado em uma ordem específica, começando com o pré-carga, permanecendo no método de atualização e chamando uma função de limpeza final se você precisar transferir para outro estado. A Phaser

> chamará esses métodos automaticamente da mesma forma que chama automaticamente a atualização do estado atual para rodar a jogabilidade única de um jogo. (Faas, 2017, p. 58)

Adicionando os estados do jogo, e carregando o estado *Boot.* O *Boot* gera a tela de abertura com o nome da Fatec e carrega o *Preloader*, responsável por carregar os arquivos que serão usados no jogo (*assets*). Como não usamos imagens, só é preciso carregar os sons e é iniciado o estado *Game*.

```
<script type="text/javascript">
    window.onload = function () {
        var game = new Phaser.Game(1280, 720, Phaser.AUTO,
'GameContainer');
        game.state.add('Boot', EmojiGame.Boot);
        game.state.add('Preloader', EmojiGame.Preloader);
        game.state.add('Game', EmojiGame.Game);
        game.state.start('Boot');
    };
```

A linguagem JavaScript é totalmente compatível com a codificação Unicode. Podemos trabalhar com os *codepoints* (código unicode correspondente) ou copiar as figuras de um site ou digitar usando o teclado virtual, inserindo os emoji diretamente no código e só não dá certo usar símbolos como nome de variáveis. Abaixo é exemplificado como as figuras foram categorizadas em vetores para invocá-los pelo índice. No caso da face do personagem que representa o status da saúde, coletar alimentos incrementa a variável que é usada também no índice do vetor *heroFace*.

```
var heroFace = new Array('💀', '💀','😠','🤯', '😠', '😭', '😢', '😣', '😨', '😂','😰',
'😢', '😳', '🤒', '😨', '😨', '😬', '😦', '😦', '😟', '😒', '😦', '😦','😐', '😶', '😌', '😚',
'😔', '🙂', '😊', '😁', '😅', '😂','😆', '😜','😛', '😜', '😖', '😍', '😘', '😙', '😷', '😗',
'😴', '😨', '🙃', '😱', '😷', '😉');
```

Outros conjuntos foram declarados da mesma forma para serem gerados aleatoriamente os itens de coleta, bichos inimigos e elementos do cenário.

```
var food = new Array("🍇", "🍈", "🍉", "🍊", "🍋", "🍌", "🍍", "🍎", "🍏", "🍐", "🍑",
"🍒", "🍓", "🍔", "🍅", "🍆", "🍄", "🍚", "🍛", "🍜", "🍝", "🍠", "🍢", "🍣", "🍤", "🍥",
  '🍡', '🍕', '🍗', '🍟', '🍦', '🍧', '🍨', '🍩', '🍪', '🍫', '🍬', '🍭', '🍮', '🍯', '🍰',
  '🍱', '🍲', '🍵', '🍶', '🍸', '🍹', '🍺', '🍻', '🍼', '🍾');
```

Abaixo o trecho de código que gera os objetos do jogo que são criados e atualizados a cada *loop* do game pelo Phaser.

```
buildWorld: function () {
    this.buildParallax();
```

```
this.buildScenery();
this.buildFood();
this.buildBugs();
this.buildHero();
this.buildPanel();
this.buildLandmarks();

music.play();
music.volume = 0.2;
if (level === 1) {
   this.menu();
   this.splash();
};
```

A seguir, o trecho que gera o personagem formado por um *sprite* vazio que define as dimensões do personagem e possui os atributos da física do jogo, onde são adicionados elementos de texto componentes da figura do herói. Sempre que muda o status da saúde ou a direção do movimento os elementos são removidos, atualizados e reinseridos.

```
buildHero: function () {
    // builds the hero
    hero = this.add.sprite(640, this.world.centerY, 'sprite');
    face = this.add.text(0, -30, heroFace[health],
    {font: '18pt', fill: "gold", stroke: "black"});
    face.anchor.set(0.5, 1);
    body = this.add.text(0, -5, "", {font: '22pt'});
    body.anchor.set(0.5, 1);
    hips = this.add.text(15, 67, "",
    {font: '15pt', fill: "DarkSlateBlue", stroke: "black"});
    hips.anchor.set(0.5, 0.5);
    leftArm = this.add.text(5, 26, "/",
    {font: 'bold 22pt Arial', fill: "gold", stroke: "black"});
    leftArm.stroke = '#000000';
    leftArm.strokeThickness = 3;
    leftArm.anchor.set(1, 0);
    leftArm.angle = 10;
    leftHand = this.add.text(0, 20, "", {font: '12pt'});
    leftHand.anchor.set(1, 0.6);
    leftHand.angle = 280;
    rightArm = this.add.text(25, 26, "\\",
    {font: 'bold 22pt Arial', fill: "gold"});
    rightArm.anchor.set(0, 0);
    rightArm.stroke = '#000000';
    rightArm.strokeThickness = 3;
    rightArm.angle = 0;
    rightHand = this.add.text(0, 20, "", {font: '12pt'});
    rightHand.anchor.set(0, 0.5);
    rightHand.angle = 70;
    leftLeg = this.add.text(6, 55, "|",
    {font: 'bold 27pt Arial', fill: "DarkSlateBlue"});
    leftLeg.anchor.set(0, 0);
    leftLeg.angle = 10;
    leftFoot = this.add.text(3, 25, "", {font: '12pt'});
    leftFoot.anchor.set(0.2, 0);
    rightLeg = this.add.text(13, 55, "|",
```

```
        {font: 'bold 27pt Arial', fill: "DarkSlateBlue"});
        rightLeg.anchor.set(0, 0);
        rightLeg.angle = 0;
        rightFoot = this.add.text(3, 25, "✍", {font: '12pt'});
        rightFoot.anchor.set(0.2, 0);
        hero.addChild(leftArm);
        leftArm.addChild(leftHand);
        hero.addChild(rightLeg);
        rightLeg.addChild(rightFoot);
        hero.addChild(leftLeg);
        leftLeg.addChild(leftFoot);
        hero.addChild(rightArm);
        rightArm.addChild(rightHand);
        hero.addChild(hips);
        hips.addChild(body);
        body.addChild(face);

        this.physics.enable(hero, Phaser.Physics.ARCADE);
        hero.body.bounce.y = 0.1;
        hero.body.bounce.x = 0;
        hero.body.gravity.y = 800;
        hero.body.gravity.x = 0;
        hero.body.drag.x = 500;
        hero.body.collideWorldBounds = true;
    },
```

Aqui apresentamos a condição de mudança de fase, quando o personagem atinge uma coordenada x definida, onde alcança uma bandeirinha vermelha como *checkpoint,* a variável level é incrementada e a fase é reiniciada, gerada aleatoriamente. A quantidade de bichos nocivos gerados inicialmente é somada com o o número da fase (*level*) fazendo com que fique cada vez mais infestado e difícil atravessar as fases.

```
  if (hero.x >= 7970) {
          if (health > 0) {
             level++;
             this.state.restart();
          };

        };

 splash: function () {
        gameName = this.add.text(100, 100, 'EMOJI', {font: '130px arial black', fill: 'gold',
align: 'center'});
        gameName.stroke = '#000000';
        gameName.strokeThickness = 22;
            gameName1 = this.add.text(100, 400, 'GAME', {font: '130px arial black', fill:
'gold', align: 'center'});
        gameName1.stroke = '#000000';
        gameName1.strokeThickness = 22;
        Splash = this.add.text(175, 210, '😀', {font: '220px arial', align: 'center'});
    },

    startGame: function (pointer) {
        score = 0;
```

```
        health = 20;
        level = 1;
        music.stop();
        this.select.play();
        this.state.start('Game');
    },

    loginGame: function (pointer) {
        this.select.play();
        if (loginEmojiGame.text === 'Login\n🔑') {
            //Chamando a DIV '#formLogin' que está dentro do index.jsp
            $('#link').click();
        } else {
            //deslogando o jogador
            document.getElementById("jogador_nome").value = '';
            document.getElementById("jogador_email").value = '';
            loginEmojiGame.text = 'Login\n🔑';
        }
        window.history.assign();
    },
    setupGame: function (pointer) {
        this.select.play();
        this.state.start('Setup');
    },

    rankingGame: function (pointer) {
        this.select.play();
        window.open('ListarRanking.jsp', "minhaJanela", "height=600,width=400");
        window.updateCommands();
    },
```

Abaixo, o trecho de código que encerra o jogo quando a variável health é zerada, aparece o texto “*Game Over*” e, ao lado, os créditos.

```
gameOver: function () {
        health = 0;
        music.stop();
        ending.play();
        over = this.add.text(150, 250, 'GAME\n\💀\n\OVER',
              {font: '150px arial black', fill: 'gray', align: 'center'});
        over.stroke = 'gold';
        over.strokeThickness = 12;
        over.fixedToCamera = true;
        over.cameraOffset.setTo(640, 350);
        over.anchor.setTo(0.5, 0.5);
        textEnd = this.add.text(0,0, credits,
        {font: '14px arial', fill: 'gold', align: 'center'});
        textEnd.stroke = '#000000';
        textEnd.strokeThickness = 4;
        textEnd.lineSpacing = -8;
        textEnd.cameraOffset.setTo(150, 250);
        textEnd.anchor.setTo(-0.1, -0.3);
        textEnd.fixedToCamera = true;

    },
```

A função update é o que faz o jogo acontecer de fato. Ela verifica a saúde do personagem, faz o cenário se mover de acordo com a posição do personagem, faz o personagem colidir com os insetos, coletar as comidas, detecta o estado das setas do teclado para mover o personagem e atualiza a posição do personagem.

```
update: function () {
        if (health === 1) {

            this.gameOver();
            this.menu();
            health = 0;

        };

        //makes the background moves slower
        if (hero.x > 640 && hero.x < 7360) {
            parallax.x = hero.x / 2;
        };

//LEVEL UP here
        if (hero.x >= 7970) {
            if (health > 0) {
                level++;
                this.state.restart();
            };

        };

        this.physics.arcade.collide(bugGroup);

        for (let i = 0; i < 20; i++)
        {
            this.physics.arcade.overlap(hero, foods[i], collect);
        }

        for (let i = 0; i < (5 + level); i++)
        {
            this.physics.arcade.collide(hero, bugGroup[i], sting);
        }

        if (cursors.left.isDown)
        {
            hero.body.velocity.x = -300;
            if (toRight === 1) {
                hero.removeChild(leftLeg);
                leftLeg = this.add.text(6, 55, "|",
                {font: 'bold 27pt Arial', fill: "DarkSlateBlue"});
                leftLeg.anchor.set(0, 0);
                leftLeg.angle = 0;
"               leftFoot = this.add.text(0, 25, "", {font: '12pt'});
                leftFoot.scale.x *= -1;
                leftFoot.anchor.set(0.6, 0);
                hero.addChild(leftLeg);
                leftLeg.addChild(leftFoot);
                hero.removeChild(rightLeg);
                rightLeg = this.add.text(13, 55, "|",
                {font: 'bold 27pt Arial', fill: "DarkSlateBlue"});
```

```
            rightLeg.anchor.set(0, 0);
            rightLeg.angle = -10;
"           rightFoot = this.add.text(0, 25, "🫲", {font: '12pt'});
            rightFoot.scale.x *= -1;
            rightFoot.anchor.set(0.6, 0);
            hero.addChild(rightLeg);
            rightLeg.addChild(rightFoot);
            hero.removeChild(leftArm);
            leftArm = this.add.text(5, 26, "/",
            {font: 'bold 22pt Arial', fill: "gold", stroke: "black"});
            leftArm.stroke = '#000000';
            leftArm.strokeThickness = 3;
            leftArm.anchor.set(1, 0);
            leftArm.angle = 0;
"           leftHand = this.add.text(0, 20, "🫳", {font: '12pt'});
            leftHand.anchor.set(1, 0.6);
            leftHand.angle = 280;
            leftArm.angle = 0;
            hero.addChild(leftArm);
            leftArm.addChild(leftHand);
            hero.removeChild(rightArm);
            rightArm = this.add.text(25, 26, "\\",
            {font: 'bold 22pt Arial', fill: "gold"});
            rightArm.anchor.set(0, 0);
            rightArm.angle = -10;
            rightArm.stroke = '#000000';
            rightArm.strokeThickness = 3;
"           rightHand = this.add.text(0, 20, "🫴", {font: '12pt'});
            rightHand.anchor.set(0, 0.5);
            rightHand.angle = 70;
            hero.addChild(rightArm);
            rightArm.addChild(rightHand);
            toRight = 0;
        };

    } if (cursors.right.isDown)
    {
        hero.body.velocity.x = 300;

        if (toRight === 0) {
            rightLeg.removeChild(rightFoot);
            leftLeg.removeChild(leftFoot);
"           leftFoot = this.add.text(3, 25, "🫲", {font: '12pt'});
            leftFoot.anchor.set(0.2, 0);
"           rightFoot = this.add.text(3, 25, "🫲", {font: '12pt'});
            rightFoot.anchor.set(0.2, 0);
            rightLeg.addChild(rightFoot);
            leftLeg.addChild(leftFoot);
            hero.removeChild(leftArm);
            leftArm = this.add.text(5, 26, "/",
            {font: 'bold 22pt Arial', fill: "gold", stroke: "black"});
            leftArm.stroke = '#000000';
            leftArm.strokeThickness = 3;
            leftArm.anchor.set(1, 0);
            leftArm.angle = 0;
"           leftHand = this.add.text(0, 20, "🫳", {font: '12pt'});
            leftHand.anchor.set(1, 0.6);
            leftHand.angle = 280;
            leftArm.angle = 10;
            hero.addChild(leftArm);
```

```
            leftArm.addChild(leftHand);
            hero.removeChild(rightArm);
            rightArm = this.add.text(25, 26, "\\",
            {font: 'bold 22pt Arial', fill: "gold"});
            rightArm.anchor.set(0, 0);
            rightArm.angle = 0;
            rightArm.stroke = '#000000';
            rightArm.strokeThickness = 3;
"           rightHand = this.add.text(0, 20, "☜", {font: '12pt'});
            rightHand.anchor.set(0, 0.5);
            rightHand.angle = 70;
            hero.addChild(rightArm);
            rightArm.addChild(rightHand);
            hero.removeChild(leftLeg);
            hero.removeChild(rightLeg);
            leftLeg = this.add.text(6, 55, "|",
            {font: 'bold 27pt Arial', fill: "DarkSlateBlue"});
            leftLeg.anchor.set(0, 0);
            leftLeg.angle = 10;
"           leftFoot = this.add.text(3, 25, "✍", {font: '12pt'});
            leftFoot.anchor.set(0.2, 0);
            rightLeg = this.add.text(13, 55, "|",
            {font: 'bold 27pt Arial', fill: "DarkSlateBlue"});
            rightLeg.anchor.set(0, 0);
            rightLeg.angle = 0;
"           rightFoot = this.add.text(3, 25, "✍", {font: '12pt'});
            rightFoot.anchor.set(0.2, 0);
            hero.addChild(rightLeg);
            rightLeg.addChild(rightFoot);
            hero.addChild(leftLeg);
            leftLeg.addChild(leftFoot);
            toRight = 1;
        };

    } if (cursors.up.isDown && hero.body.onFloor() && this.time.now > jumpTimer)
    {
        jump.play();
        jump.volume = 0.5;
        hero.body.velocity.y = -700;
        jumpTimer = this.time.now + 750;
    }

    function collect(hero, food) {
        if (health < 39) {
            health++;
        };

        scoreScreen.text = heroFace[health] + ++score;
        face.text = heroFace[health];
        body.removeChild(face);
        body.addChild(face);
        katch.play();
        katch.volume = 0.2;
        food.kill();
    }

    function sting(hero) {
        if (health > 1) {
            health--;
        };
```

```
            scoreScreen.text = heroFace[health] + score;
            face.text = heroFace[health];
            body.removeChild(face);
            body.addChild(face);
            hurt.play();
        }

        if (this.instructions.exists && this.time.now > this.instExpire) {
            this.instructions.destroy();
        };

        if (document.getElementById("jogador_nome").value !== '') {
            var nome = document.getElementById("jogador_nome").value;
n           loginEmojiGame.text = nome.substr(0, 10) + '\n🔑';
        } else {
n           loginEmojiGame.text = 'Login\n🔑';
        }

    }//Update ends here
};
```

3.6.1 Criação de ativos (*game assets*)

Os sons utilizados foram obtidos de arquivos gratuitos, disponibilizados em sites dedicados a oferecer sons para games. Não houve nenhuma necessidade de criar desenhos nem de adquirir imagens pois toda a parte gráfica vem de elementos de texto da fonte padrão do navegador.

3.6.2 *Debugging, Tuning* e polimento

A cada pequeno avanço no desenvolvimento do protótipo realizado pelo desenvolvedor principal do jogo, o código era apresentado para a equipe que estudava, efetuava testes, e sugeria melhorias. Houve muitas dificuldades em adaptar as funções da *engine* Phaser para a tecnologia proposta de só trabalhar com texto. Várias características apresentavam efeitos colaterais ou simplesmente não funcionavam. Vários *bugs* foram resolvidos pelos membros da equipe e não teríamos conseguido terminar o jogo sem o empenho dos participantes em ajudar na programação. Além disso, programaram todo o sistema de cadastro de usuários e pontuações salvos em Java no banco de dados PostgreSQL, integrados à interface do jogo.

3.6.3 Implementação

O jogo foi publicado pelos membros da equipe responsáveis pelo desenvolvimento do *back-end* e está disponível, hospedado no Azure da Microsoft.

3.7 Pós-produção

Após a publicação do protótipo e apresentação para o público, encerra-se um ciclo de desenvolvimento com a produção desse Trabalho de Conclusão de Curso, que cumpre a função de documentação do ciclo passado. Também começa um novo ciclo de desenvolvimento com elaboração de um Documento de Projeto de Jogo ou Game Design Document (Apêndice 1), Plano de Negócio (Apêndice 2) e modelo canvas (Apêndice 3).

4. RESULTADOS E IMPLICAÇÕES

Obtivemos sucesso na empreita acadêmica de criar dentro do prazo todas as partes do projeto, incluindo a criação dos documentos de especificação, o desenvolvimento do *software*, hospedagem no Azure da Microsoft e apresentação para professores e profissionais convidados com bom retorno e avaliação por parte do público. O protótipo também foi apresentado em comunidades de desenvolvedores nas redes sociais e os retornos com as opiniões foram quase todas positivas quanto a tecnologia apresentada e conceito gráfico, mas todos pediram por melhorias na animação, interações, melhor definição dos objetivos e mais opções de jogabilidade.

A seguir, apresentamos capturas de tela do protótipo implementado que chamamos de EMOJIGAME. Com as setas direcionais do teclado o jogador movimenta o personagem para os lados e pula para agarrar alimentos que ficam rebatendo pelo cenário ou desvia de insetos que ficam voando pela tela. Ao conseguir atravessar a fase gerada aleatoriamente o jogo muda de fase, com cada vez mais inimigos para dificultar o percurso. Pegando alimentos, o personagem vai ficando com uma expressão mais positiva e quando toca nos insetos ocorre o efeito contrário. Se a condição de saúde, representada pela expressão no rosto do personagem, chega ao zero, representado por uma caveira, o jogo acaba.

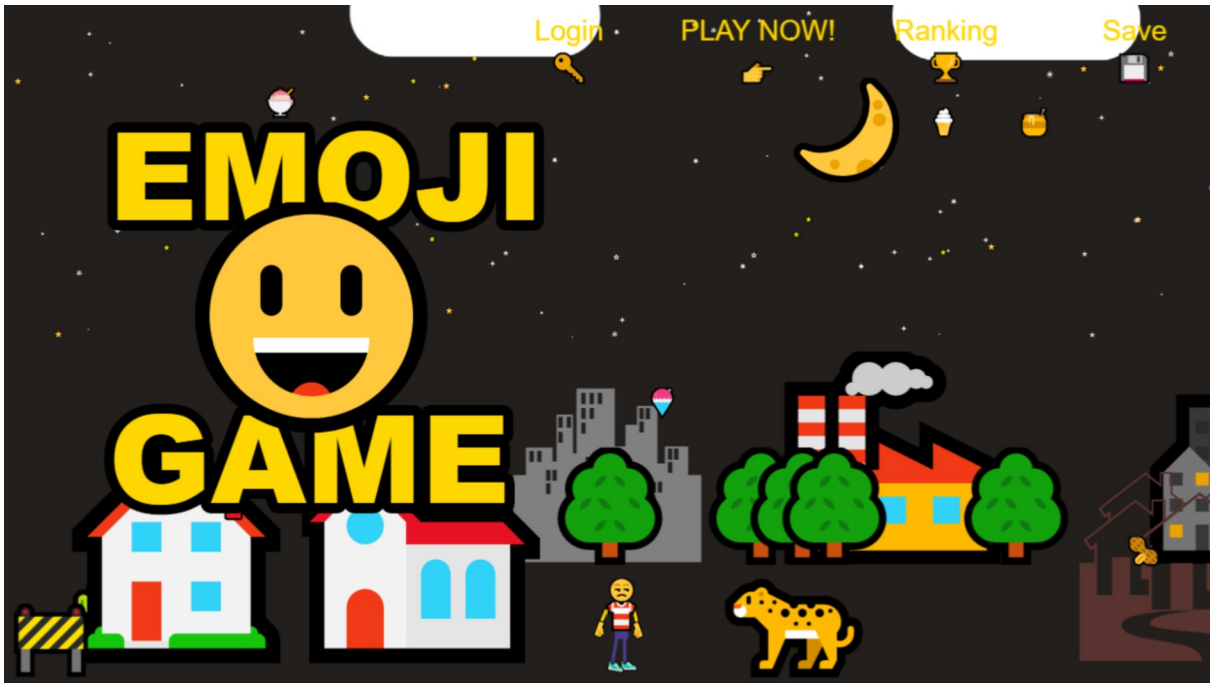
Existe a opção de se cadastrar para salvar o resultado a partida, apresentado em um ranking de melhores jogadores, mas a priori o jogo já carrega em uma partida, convidando o jogador a explorar o *gameplay* e descobrir intuitivamente os objetivos.

Figura 9: Detalhe de tela do jogo, mostrando o personagem.



Fonte: autor.

Figura 10: tela de abertura com splashscreen à esquerda e menu acima.



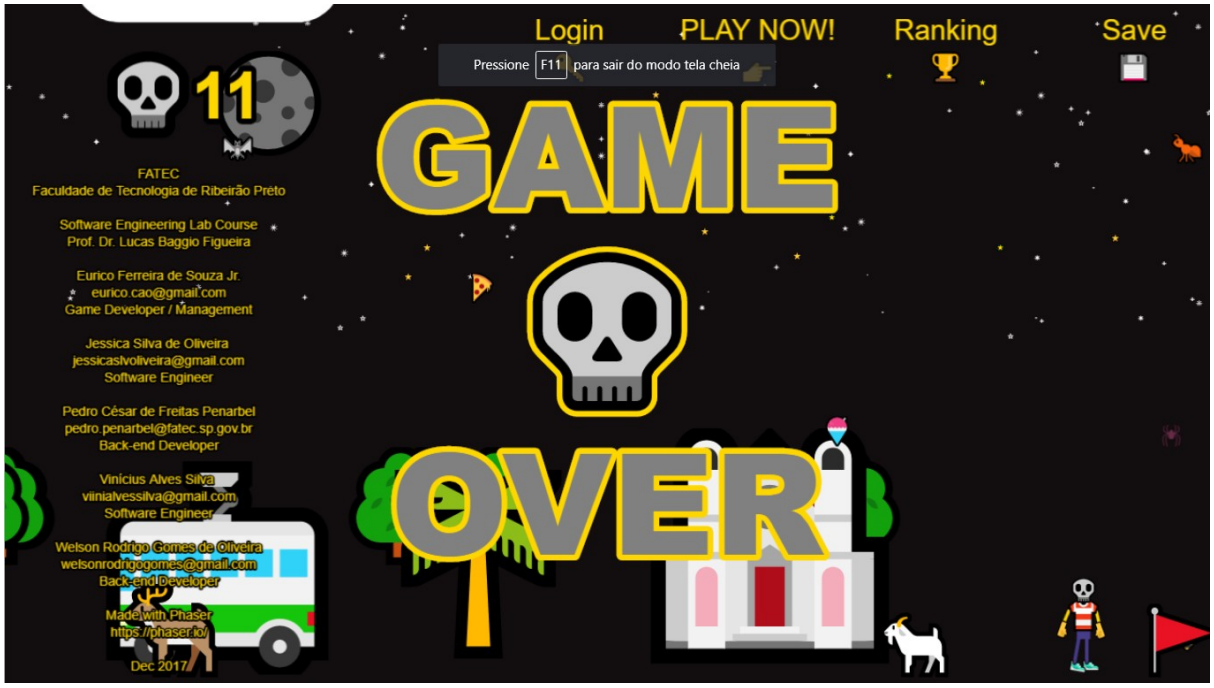
Fonte: autor.

Figura 11: Acima à esquerda o placar que repete a expressão do personagem



Fonte: autor.

Figura 12: Tela de fim do jogo., onde aparecem os créditos e o menu.



Fonte: autor.

5. SUMÁRIO, DISCUSSÃO E CONCLUSÕES

5.1 Discussão dos resultados

Pelos resultados obtidos nesse projeto, pudemos observar que o componente canvas do HTML5 oferece liberdade e recursos para composição artística com uso de elementos de texto, como os emoji. A linguagem JavaScript é amigável com a codificação unicode e os navegadores mostraram boa performance, animando vários elementos com o uso da engine Phaser, indicando que sim, é possível fazer um *game* de ação estilo plataforma *arcade* baseado em texto, sem utilização de nenhuma imagem, desenho vetorial ou modelo 3D, em que todas imagens são formadas com fontes que já estão no computador do usuário.

Os métodos para atualização de atributos do *emoji* que representavam a cabeça, posição e ângulos das partes do personagem, conforme a *engine* Phaser, não funcionaram como previsto e tivemos que remover e reinserir outros elementos iguais em outra posição a cada atualização para obtermos as mudanças. Essa dificuldade impediu-nos de usarmos vários recursos da *engine* e não deu tempo de implementar os movimentos do corpo do personagem nessa versão do game.

Não conseguimos fazer o full-screen (tela cheia, sem a barra de endereços do navegador) funcionar no navegador Edge, mobile, nem no Safari. Para a próxima versão, devemos tentar melhorar a diagramação, para que não apareça a barra de rolagem, nem corte o personagem.

Alguns emoji não são mostrados no Chrome e o conjunto completo só funcionou no Edge. O game funciona muito bem no Mac, mas a apresentação dos emoji é diferente. No Firefox e no Linux, as figuras coloridas não aparecem e, como não houve tratamento para colorir os elementos de texto, o resultado ficou insatisfatório.

As tecnologias JavaEE e JSF que aprendemos em programação para web e utilizamos para desenvolver o back-end não trabalham com o canvas do HTML5, então as telas de login e ranking não puderam ser integralizadas e aparecem em telas separadas.

Voltando ao primeiro objetivo desse estudo, sobre a possibilidade se fazer jogos de ação modo texto, o protótipo do EMOJIGAME indica que, pelo menos para

web games desenvolvidos com JavaScript, a proposta parece ser plenamente viável em computadores, mas necessita de mais estudos para ser viabilizada para *mobile,* por ter apresentado baixa performance, com movimentos travando. Em versões antigas do Android, o jogo nem abriu.

Outros objetivos propostos não puderam ser alcançados, porque o protótipo ainda está muito rudimentar para ser apresentado comercialmente e precisamos lançar um bom jogo para aumentar a visibilidade da técnica. A necessidade de refatoração para nova versão do Phaser e implementação de novas jogabilidades adiou a implementação do versionamento e disponibilização do projeto no GitHub.

5.2 Implicações para a prática

A demonstração da versatilidade e conveniência de usarmos os emoji, símbolos e caracteres unicode como ativos gráficos em jogos digitais pode facilitar e fomentar o uso da criação de games em cursos de programação, a formação de criadores de jogos e incentivar a retomada da produção de jogos em modo texto. A tecnologia pode ser aplicada além dos games, em animações, infográficos, propagandas, apresentações, treinamentos, contação de histórias, web design, design de interfaces, e outros, pois permite facilmente editar e customizar os conteúdos e os arquivos ficam bem menores que arquivos de imagem e vídeo.

A utilização dos emoji e letras para composição de personagens no contexto diferenciado apresentado no protótipo EMOJIGAME pode gerar Propriedade Intelectual e agregar valor na utilização dos conteúdos produzidos em outros jogos e produtos licenciados.

5.3 Recomendações para pesquisas futuras

É muito importante melhorar a acessibilidade, incluindo a possibilidade de customizar os controles do teclado, permitir o controle por joystick e implementar o j*oystick* virtual (touch API), para permitir que o jogo funcione em dispositivos mobile. Também é interessante implementar interface distribuída, com *joystick* na tela do celular, para jogar na smart TV.

O jogo está no estágio de protótipo e naturalmente precisa ser incrementado. O primeiro passo será reavaliar a conveniência do uso do Phaser. Se sim, refatorar o jogo para a nova versão 3 (3.11), reconstruída e incompatível com a usada no protótipo e para melhoria da arquitetura do sistema e adequação o código a novas ideias de jogabilidade, incluindo animar e incluir novas ações para o personagem, inserir personagens do jogo (NPCs), melhorar o *gameplay*, produzir fases consistentes e música própria. O potencial de evolução do sistema inclui a possibilidade de cadastro pelas redes sociais, criação de jogos *multiplayer,* geração procedural de fases, uso de ferramenta de análise do comportamento e perfil dos usuários como o Google Analytics. Buscar melhorar a exibição dos *emoji* em versões anteriores do Windows, Linux e navegador Firefox através da definição de cores para os elementos gráficos que normalmente são coloridos funcionarem bem com sistemas de fontes monocromáticas.

Após um ciclo de desenvolvimento futuro, propomos a criação de campanha de *marketing*, implementação de mecanismos de acesso pelas redes sociais, tradução da documentação e material de apoio, publicação em servidor profissional (o azure para estudantes, onde o protótipo está publicado, é bem limitado) e criação de vídeos, textos e outros materiais de divulgação.

Uma ideia para a continuação do desenvolvimento do projeto de forma colaborativa é a proposta de gamificação da produção de conteúdos para o jogo, com pagamentos e premiação efetuados com participação na propriedade e resultados financeiros. Nesse modelo de desenvolvimento, seria criado um ambiente de apoio online anexo ao game, no qual os usuários cadastrados teriam acesso ao material de apoio para sua capacitação como desenvolvedores, e escolheriam num painel estilo *kanban,* dentre as atividades disponíveis a serem feitas, o que querem pesquisar e produzir. As tarefas realizadas pelos usuários aprovadas para serem implementadas no jogo seriam ser pagas com criptomoeda própria, o EmojiCoin. Emojicoins, que poderiam ser usados para compra de itens no jogo, representariam ações do jogo, seriam títulos de honra e poderiam dar poderes especiais no jogo ao portador. Investidores participariam adquirindo EmojiCoins. O jogo continuaria gratuito para jogar, mas para salvar EmojiCoins encontrados ou ganhos em competições, o jogador precisará de adquirir uma carteira digital, ou EmojiWallet, uma conta *premium* com acesso a material de apoio e sistemas de administração do

jogo. Adquirir uma conta *premium* seria então interessante para jogadores, investidores e estudantes.

O projeto pede mais estudos para elaboração de orçamento de custos, melhoria no plano de negócio, a criação de um sistema de gerenciamento automatizado da administração do jogo, incluindo o sistema de compra de assinaturas e itens do jogo, a elaboração de um plano de marketing com produção de material de divulgação, análise e gerenciamento de riscos, e todos os outros aspectos administrativos ligados à equipe de Gestão de Negócios e Inovação da Fatec.

Futuramente poderemos organizar os conhecimentos adquiridos na forma de um curso de programação através do desenvolvimento de jogos para nivelamento de novos colaboradores e interessados em participar do projeto ou desenvolver outros. Os processos, conhecimentos e tecnologias poderiam ser empregados na produção e administração de outros projetos de games.

Outras ideias relacionadas ao game design, enredos e *gameplay* para serem implementadas no EMOJIGAME são apresentadas no Documento de Projeto de Jogo ou, como é mais conhecido, *Game Design Document,* o GDD (apêndice 1).

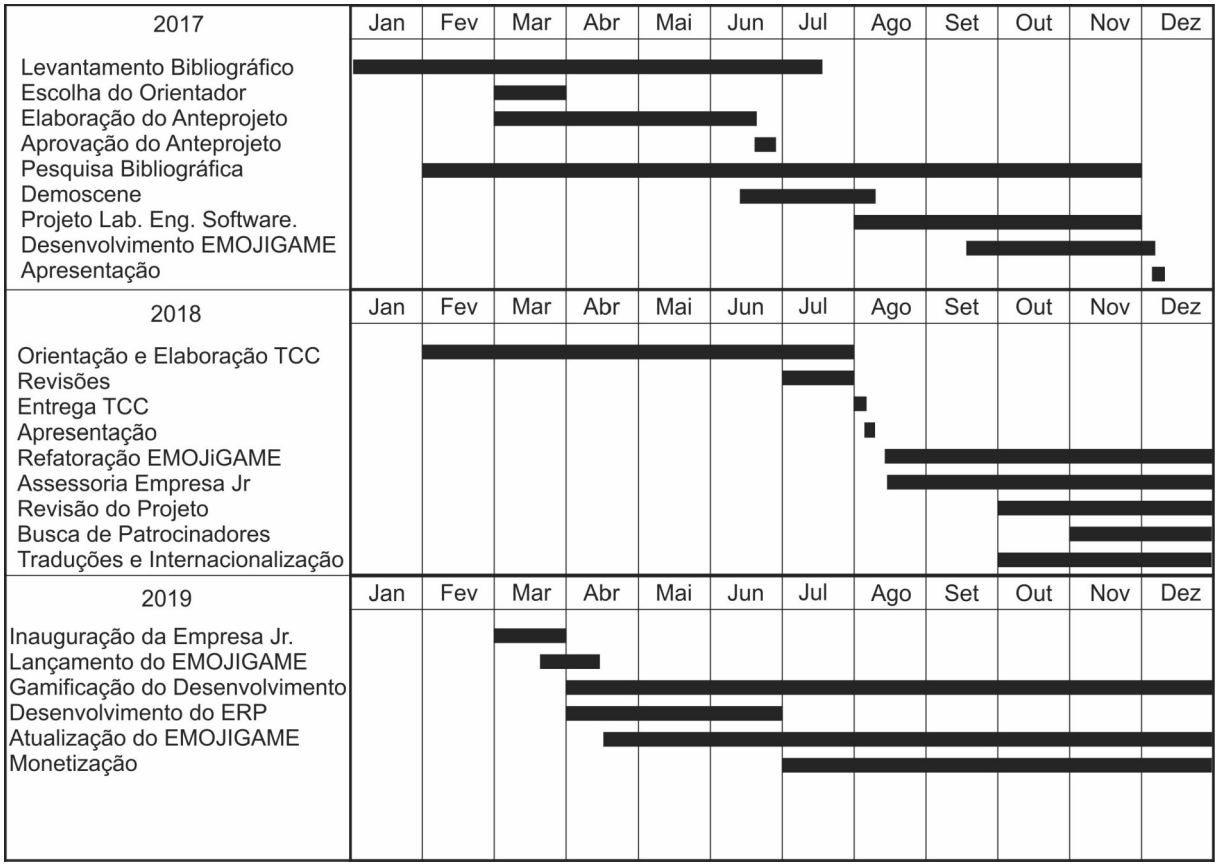
5.4 Sumário e Considerações finais.

A primeira turma do período diurno do curso de Análise e Desenvolvimento de Sistemas da Fatec Ribeirão Preto, valendo-se das tecnologias aprendidas durante o curso, especificou, projetou, implementou, testou e apresentou um protótipo de web game de ação que usa somente emoji e outros elementos de texto na codificação unicode como elementos gráficos, apresentando a inovação que simplifica o processo de desenvolvimento. Pelos resultados obtidos podemos concluir parcialmente, respondendo sim à pergunta da pesquisa, demonstrando que é possível usar emoji e outros elementos de texto na codificação unicode como ativos gráficos para a produção de games de ação. O experimento esteve limitado a um prazo de execução de poucos meses, tivemos problemas ao adaptar a técnica proposta às funções da engine Phaser e sentimos a falta de exemplos parecidos nos tutoriais. O projeto indica ter potencial de desenvolvimento e pede novos estudos e

produção de conteúdo para que possamos atrair colaboradores, criar capital intelectual e propriedade industrial, lançar e administrar o EMOJIGAME como empreendimento financeiramente sustentável e plataforma para a formação de desenvolvedores de jogos e também disseminar o uso da tecnologia por outros desenvolvedores com o objetivo final de desenvolver a economia e disseminar o conhecimento.

6. CRONOGRAMA

Quadro 1: Cronograma Geral



Fonte: autor.

7. REFERÊNCIAS BIBLIOGRÁFICAS

720P. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=720p&oldid=52451057>. Acesso em: 28 jul.. 2018.

ADAMS, Ernest; DORMANS, Joris. *Game Mechanics. Advanced game design.*

Berkeley: New Riders Games, 2012.

BATES, Bob. *Game Design.* 2nd ed. Boston: Thomson, 2004.

BATES, Bob. *Game Developer's market guide.* Boston: Premier Press, 2015.

BEST open source javascript. Our Code World. Disponível em: <https://ourcodeworld.com/articles/read/308/top-15-best-open-source-javascript-game-engines>. Acessado em 20 jul. 2018.

BETHKE, Erik. Game Development and Production. Plano: Wordware Publishing, 2003.

BIBAT, Bryan. HTML 5 Shoot 'em Up in an Afternoon. Disponível em:<https://leanpub.com/html5shootemupinanafternoon/read> Acessado em 29 Jul. 2018.

BORRIES, Friedrich von; Walz, Steffen P.; BÖTTGER, Matthias. *Space Time Play. Computer Games, Architecture and Urbanism: The Next Level.* Berlin: Birkhäuser, 2007.

BOURG, David M.; BYWALEC Physics for *game developers*. Sebastopol: O'Reilly, 2013.

BROWSER GAME. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2016. Disponível em: <https://pt.wikipedia.org/w/index.php?title=Browser_game&oldid=47473209>. Acesso em: 16 julho 2018.

BUCK, Jamis. Mazes for Programmers – Code Your Own Twist Little passages. The Pragmatic Bookshelf, 2015.

BUNYAN, Karl. *Build an HTML5 game. A Developer's Guide with CSS and JavaScript.* San Francisco: No starch press, 2015.

CÁSSIO, Éderson. Desenvolva jogos com HTML5 e Canvas. Casa do Código.

CHILDS, G. W. IV. *Creating Music and Sound for Games.* Boston: Thomson, 2007.

COLLINS, Karen. Game Sound. MIT Press, 2008.

COHEN, D. S.; BUSTAMANTE, Sergio. A. II. *Producing Games. From business and budgets to creativity and design*. Burlington: Elsevier, 2010.

COOPER, Kendra M. L.; SCACCHI, Walt (Ed.). *Computer Games and Software Engineering*. Boca Raton: CRC Press, 2015.

CRAWFORD, Chris. *On Game Design.* New Riders Publishing, 2003.

CRAWFORD, Chris. *The Art of Computer Game Design.* Vancouver: Washington

State University, 1997.

DAVEY, Richard. *Getting Started with Phaser 3. Part 2 – Installing the Web Server*. Disponível em <http://phaser.io/tutorials/getting-started-phaser3/part2>. Acessado em 12 Jul. 2018.

DESPAIN, Wendy. (Ed.) *100 Principles of Game Design.* New Riders, 2013.

DYRR, Kristen et all. *Game Development for Human Beings – Build Cross-Platform Games With Phaser*. Zenva, 2016.

DOBRE, Adrian; RAMTAL, Dev. *Physics for JavaScript Games, Animation, and Simulation.* Apress, 2014.

EGGES, Arjan. Building JavaScript Games for Phones, Tablets and Desktops. Apress, 2014.

EMOJI. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=Emoji&oldid=52607139>. Acesso em: 159 jul. 2018.

FAAS, Travis. An Introduction to HTML5 Game Development with Phaser.JS. CRC Press, 2017.

FEIL, John; SCATTERGOOD, Marc. *Beginning Game Level Design.* Boston: Thomson, 2005.

FLANAGAN, Mary. *Critical Play. Radical Game Design*. Massachusetts: MIT Press, 2009.

FLYNT, John P.; SALEM, Omar. *Software Engineering for Game Developers*. Boston: Thomson, 2005.

FONTE tipográfica. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=Fonte_tipogr%C3%A1fica&oldid=52654216>. Acesso em: 13 jul. 2018.

FOX, Brent. *Game Interface Design.* Boston: Thomson, 2003.

FREQUENTLY asked questions. Unicode.org. In: <https://www.unicode.org/faq/basic_q.html> Acesso em: 15 julho 2018.

FULLERTON, Tracy; SWAIN, Christopher; HOFFMAN, Steven. *Game Design Workshop: A playcentric approach to creating Innovative Games.* 2nd ed. Darlington: Elsevier, 2008.

FULTON, Steve; FULTON, Jeff. HTML5 Canvas. O'reilly, 2013.

GEARY, David. *Core HTML5 Canvas. Graphics, Animation and Game Development.* Prentice Hall, 2012.

GOLD, Julian. *Object-oriented Game Development.* Essex: Pearson, 2004.

GOSE, Stephen. Phaser Game Prototyping. Disponível em: <>. Acesso em 14 Jul. 2018.

HARRIS, Andy. HTML5 Game Development for Dummies. New Jersey. Willey. 2013.

HOW TO debug phaser games. Code academy. Disponível em:<https://gamedevacademy.org/how-to-debug-phaser-games/>. Acessado em 12 de julho de 2018.

HTML5 canvas. In: w3schools.com. Disponível em <https://www.w3schools.com/html/html5_canvas.asp>. Acesso em 15 de julho de 2018.

IRISH, Dan. *The Game Producer's handbook.* Boston: Thomson, 2005.

ISBISTER, Katherine. *Better Game Characters by Design. A psychological approach.* San Francisco: Elsevier, 2006.

ISBISTER, Katherine. *How Games Move Us. Emotion by design.* Massachusetts: MIT Press, 2006.

JAVASCRIPT. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=JavaScript&oldid=51754799>. Acesso em: 15 de julho de 2018.

JUUL, Jesper. *A Casual Revolution. Reinventing Video Games and Their Players.* Massachusetts: MIT Press, 2010.

KEITH, Clinton. *Agile Game Development with SCRUM.* Boston: Pearson, 2010.
KOSTER, Raph. *A Theory of Fun for Game Design.* Scottsdale: Paraglyph Press, 2005.

KURIANOVICH, Egor. HTML5 Games Most Wanted. Build the best HTML5 Games. Apress, 2012.

McANLIS, Colt (et. al.). *HTML5 Game Development Insights*. Berkeley: Apress, 2014.

MAKING your first html5 game with phaser. Tutorialzine. Disponível em: <https://tutorialzine.com/2015/06/making-your-first-html5-game-with-phaser> Acessado em 29 Jul. 2018.

MANIFESTO ágil. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2017. Disponível em: <https://pt.wikipedia.org/w/index.php?title=Manifesto_%C3%81gil&oldid=48667990>. Acesso em: 25 julho 2018.

MARKS, Aaron. *The Complete Guide to Game Audio for Composers, Musicians, Sound Designers, and Game Developers*. 2nd Ed. Oxford: Elsevier, 2009.

MAYER, Andrew. *Making games better. The Art and Process of Game Design and Development*. LevelUpDesign, 2016.

MEYER, Jeanine. Using Games to Learn HTML5 and JavaScript. Apress, 2010.
MOORE, Michael E. *Basics of Game Design.* Boca Raton: CRC Press, 2011.

MOTOR de jogo. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=Motor_de_jogo&oldid=51652086>. Acesso em: 16 Julho. 2018.

NOVAK, Jeannie. *Game Development Essentials. An Introduction*. 3rd ed. Clifton Park: Cengage Learning, 2012.

OXLAND, Kevin. *Gameplay and dDesign.* Essex: Pearson, 2004.

O'NEIL, Harold F.; PEREZ, Ray S. *Computer Games and Team and Individual Learning*. Elsevier, 2008.

PALMER, Grant. Physics for Game Programmers. Berkeley: Apress, 2005.

PEDERSEN, Roger E. *Game design foundations.* Plano: Wordware Publishing, 2003.

PERRY, David; DEMARIA, Rusel. *Game design. A Brainstorming Toolbox.* Boston: Cengage Learning, 2009.

PRESSMAN, Roger S. Engenharia de Software . ABDR, 2011.

RETTIG, Pascal. *Professional HTML5 Mobile Development.* Willey, 2012.

ROGERS, Scott. *Level Up! The guide to create video Game Design.* Willey, 2010.

ROLLINGS, Andrew; MORRIS, Dave. *Game Architecture and Design: A new edition.* Indianapolis: New Riders, 2004.

ROLLINGS, Andrew; ADAMS, Ernest. *On Game Design.* Indianapolis: New Riders Publishing, 2003.

ROUSE, Richard III. *Game design theory and practice.* 2nd ed. Plano: Wordware Publishing, 2005.

ROWELL, Eric. HTML5 Canvas Cookbook. Packt, 2011.

RUCKER, Rudy. *Software engineering and computer Games.* Essex: Addison Weley, 2002.

SCHELL, Jesse. *The art of Game Design. A book of lenses. 2 ed.* Burlington: Elsevier, 2015.

SHANKAR, Adytia Ravi. Pro HTML5 Games. Apress, 2012.

SHELDON, Lee. *Character development and storytelling for Games*. Boston: Thomson, 2004.

SIDE-scrolling video game In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://en.wikipedia.org/wiki/Side-scrolling_video_game> Acesso em 18 Julho 2018.

SMUS, Boris. Web Audio API. O'reilly, 2013.

SOMMERVILLE, Ian *Software engineering* . 9th ed. Pearson, 2011.

SPUY, Rex Van Der. Foundation Game design with HTML5 & JavaScript. Apress, 2012.

STEINBERG, Scott. Videogame Marketing and PR. Power Play Publishing, 2007.
SWINK, Steve. *Game Fell. A Game Designer's Guide to Virtual Sensation.* Burlington: Elsevier, 2009.

SILVEIRA, Rodrigo. *Multiplayer Game Development with HTML5.* Packt Publishing 2015.

SYLVESTER, Tynan. *Designing Games. A Guide to Engineering Experiences.* Sebastopol: O'reilly, 2013.

TAN, Philip; EBERHARDT, Richard; VERRILLI, Sara; and GRANT, Andrew *CMS.611J Creating Video Games.* Fall 2014. Massachusetts Institute of Technology: MIT OpenCourseWare https://ocw.mit.edu. License: Creative Commons BY-NC-SA. Disponível em: <https://ocw.mit.edu/courses/comparative-media-studies-writing/cms-611j-creating-video-games-fall-2014/index.htm#> Acessado em 10 Jul. 2018.

TEXT-based GAME. In *Wikipedia, The Free Encyclopedia*. disponível em: <https://en.wikipedia.org/w/index.php?title=Text-based_game&oldid=807434714>. Acessado em: 16 julho 2018. tradução nossa.

TEXT mode The Free Encyclopedia, 30 April 2018, 18:21 UTC, disponível em: <https://en.wikipedia.org/w/index.php?title=Text_mode&oldid=839014385> Acessado em: 22 julho 2018. tradução nossa.

TREFAY, Gregory. *Casual Game Design. Designing play for the gamer in all of us.* Burlington: Elsevier, 2010.

UML. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=UML&oldid=51867830>. Acesso em: 15 de julho de 2018.

UNICODE. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2018. Disponível em: <https://pt.wikipedia.org/w/index.php?title=Unicode&oldid=52053395>. Acesso em: 15 julho 2018.

WALKER, Mark H. *Games that Sell!* Plano: Wordware Publishing, 2003.

WATSON, John. *Game Mechanic Explorer.* Disponível em: <https://gamemechanicexplorer.com/>. Acessado em 29 Jul. 2018.

WILLIAMS, James L. *Learning HTML5 Game Programming. A hands-on guide to building online games using canvas, SVG, and WebGL.* Boston: Pearson Education, 2012.

Apêndice 1 – DOCUMENTO DE PROJETO DE JOGO (*Game Design Document*)

Background

O documento de projeto de jogo, ou GDD, é um documento de design vivo (sempre em atualização) altamente descritivo do design de um videogame para organizar esforços dentro de uma equipe de desenvolvimento e apresentar o projeto para investidores.

A seguir apresentamos uma segunda versão, ainda em desenvolvimento e incompleta, do documento de projeto de jogo (a primeira fez parte da documentação produzida no laboratório de engenharia de software) trazendo mais ideias e diretrizes para prosseguir com o desenvolvimento do EMOJIGAME.

Descrição (Headline)

Web game de ação casual, de plataforma tipo *side scrolling runner*, com vários minigames inspirados em video games clássicos incluídos em fases bônus, todo feito em modo texto com a temática dos emoji, em que o personagem reage às situações do jogo com as diversas expressões faciais dos emoji *smiles* e gestos das mãos emoji.

Características Principais

EMOJIGAME é um Jogo simples e casual, autoexplicativo, que explora os clichês dos jogos clássicos em que o jogador identifica o objetivo através de referências a situações típicas dos games de 8 e 16 bits, a experiência que o jogo propicia é a ambientação e situações em homenagem aos clássicos arcade e dos consoles 8bit *oldschool.* O jogador é surpreendido a todo momento com o surgimento de novas fases com mudança de gameplay, mecânica do jogo, objetivos, como se tivesse saído de um game e entrado em outro. Dessa forma as equipes de

desenvolvimento colaborativo podem oferecer suas participações com a criação de novos conteudos que são adicionados ao game, deixando em aberto as possibilidades de expansão e desenvolvimento do projeto.

Gênero

Jogo 2D de ação casual estilo arcade.

Arte conceitual

Projeto do Jogo

Mecânica do Jogo

Personagens articulados estilo boneco de palito (*ragdoll*) montados inteiramente com letras, símbolos e EMOJI do UNICODE com expressão facial e corporal dinâmicas, realizadas comutando as diversas possibilidades de faces e mãos do repertório disponível, capaz de efetuar troca de roupa alterando as cores, e mudar acessórios e objetos que empunha ou carrega.

Ambientado em um mundo povoado pelos objetos, edifícios, veículos, animais e outros itens dos EMOJI ou compostos por letras e símbolos. O jogo não usa bitmaps nem a tecnologia de sprites, o que diferencia a estética e aumenta as possibilidades de efeitos de animação.

Homenageia, faz referência e brinca com os clichês, atmosfera e estética dos jogos retrô 8 bit, com gameplay independente e variável em cada fase, passando pelos jogos sem perspectiva, corredores de plataforma, isométrico e até pseudo3D na intensão de reproduzir a experiência e sensação de jogar antigos jogos estilo Atari, Mega Drive, Nintendo e Genesis, com as imagens sendo formadas por símbolos e texto.

Core Game Play

Inspirado pelo conteúdo dos emoji, na quantidade de animais, profissões, alimentos, objetos, e corações, e na proposta do jogo ser coleção de clones de cássicos, os objetivos do jogo são o contrário do que geralmente acontece nos games. O objetivo é agradar os animais com os alimentos coletados, arrumar e consertar o ambiente, ao invés de destruir. Fazer amizade com os personagens do jogo e agradar o chefe da fase.

O cenário principal é uma plataforma 2D gerada proceduralmente e tem vários temas: selva, fazenda, cidade, fase aquática, deserto, etc, (organizando e melhorando o cenário aleatório apresentado no protótipo EMOJIGAME e

acrescentando vários outros pictogramas que não foram utilizados), veículos e edifícios podem ser entrada para fases bônus, com minigames de mecânica simples e curta duração. O objetivo é coletar itens e cumprir missões para ter acesso a novas fases com gameplay mais elaborado, com menos reação imediata e mais estratégia e interações estilo RPG, quebra-cabeças, como evoluiu a história dos games. A mudança de fase faz com que objetos tenham funções diferentes, sendo vitais ou inúteis, ofereçam perigo ou não, dependendo de que fase se está. O nível ou habilidade adquirido em uma fase talvez seja irrelevante em outra de contexto diferente. Os comandos aplicáveis também variam de acordo com o contexto da fase. Se necessário, regras escritas aparecem sobrepostas à cena do jogo. O ponto de vista pode variar e os gráficos, por serem formados por texto, são naturalmente escalonáveis, podendo apresentar o personagem pequeno como no Warcraft ou maior como em jogo de luta.

Game Flow

O fluxo do jogo tem o estilo dos sandbox games, ou mundos abertos, sem a obrigação linear de uma sequência de missões ou ameaças, e pode ser descrito como um “parque de diversões” de minigames retrô onde o jogador experimenta e explora a mecânica de jogos primitivos em incursões rápidas. O desafio principal do jogo é estar bem ranqueado entre os jogadores e adquirir ítens.

Explorando as fases, interagindo com NPCs, veículos e objetos, coletando itens, acumulando pontos, aprendendo (adquirindo) comandos, o jogador acumula pontos e itens coletados. Ao final da partida é oferecido o registro para ter o game salvo para quem jogou sem ter se logado. É desejável que o jogo seja capaz de fazer cadastro pelas principais redes sociais.

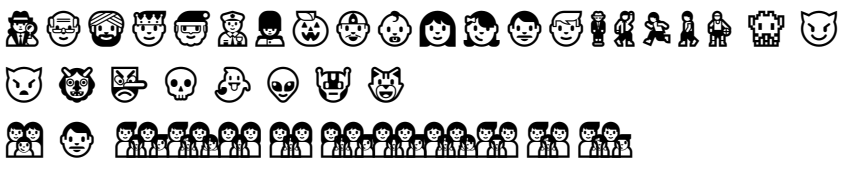
Personagens

O boneco de palito com a face de emoticon é naturalmente impessoal, ficando a caracterização restrita à cor de roupa, acessórios e pequenas variações no biótipo (tamanho das partes do corpo podendo fazer variar altura e porte físico dos bonecos). O projeto pede a criação de um personagem feminino para ser escolhido e, pelo que a disponibilidade sugere, um gato ajudante com várias expressões faciais. Podemos criar uma narrativa que aproveite os personagens dos EMOJI (ver exemplos no ítem NPCs a seguir) e seu universo em expansão (todo ano são publicados novos EMOJI e vários dos símbolos já existentes estão marcados para serem convertidos em desenhos coloridos)

Elementos do Jogo

Basicamente o personagem pode correr, pular, golpear ou usar ferramentas e coletar itens (comida e bebida são consumíveis ou podem entrar no inventário) no mundo do jogo que contém as entradas para os mini games. Dentro dos mini games o personagem adquire interações específicas da fase.

NPCs, os Personagens não jogáveis: podem ter diversas faces diferentes. Pequenas figuras podem ser usadas para representar pessoas do cenário.



Edificações: são as entradas para as fases e devem ser exploradas.



Veículos: alguns podem ser pilotados e representam a entrada para a fase.



Alimentos e bebida: Restauram a saúde ou tempo extra.



Dinheiro e outros objetos de valor: Pontuação e compra de itens.



Troféus e medalhas: Ganhas no cumprimento de missões.



Ferramentas:



Armas:



Animais (NPC):



Elementos de cenários:





Acessórios:



Remédio e itens de cura:



Combustível:



Sinalização:



Objetos diversos:



Física do Jogo e Estatística.

A física do jogo é baseada em animações estilo arcade inspiradas no estilo dos jogos 8 bit e desenhos animados (cartoon), sem compromisso com simulação de interação entre objetos, apenas detecção de colisão entre o personagem e objetos e piso (quando aplicável).

Multiplayer

Por enquanto a preocupação é não criar um modelo de mundo que dificulte a futura implantação de multiplayer e o estímulo a competição mostrando durante o jogo o momento que ultrapassa o progresso e pontuação de outros jogadores e a posição do ranking.

Interface do usuário

O jogo será apresentado em modo de tela cheia, apresentando apenas o personagem e o mundo do jogo, placar, inventário, mapa, instruções e outros são eventualmente apresentados momentaneamente e sobrepostos à cena do jogo.

Flowchart

Requisitos Funcionais

O jogador usa as setas direcionais do teclado e barra de espaço para ação básica, outros botões podem ser ocasionalmente ativados e aparece na tela

instrução avisando quais botões usar. (Na versão mobile, a ser futuramente desenvolvida, os botões aparecem na tela conforme a necessidade).

Maquetes

Arte e Vídeo

As animações são todas feitas diretamente no elemento Canvas do HTML5 controlado com JavaScript. Os sons e música podem ser arquivos de áudio ou gerados no Web Audio API. É desejável que em versões futuras seja introduzido diálogos em interações mais elaboradas, podendo ser no formato texto ou usando gerador de voz, conforme o estilo de jogo da fase pedir.

Metas Gerais

Publicar em servidor um Alfa que tenha ao menos três mini games para demonstrar a dinâmica do jogo e o funcionamento do back end e banco de dados.

Arte 2D e Animação

Usando os comandos de manipulação de elementos do Canvas podemos rotacionar, redimensionar (mudar o tamanho da fonte), usar efeitos de distorção,

Vários efeitos são aplicáveis ao texto nos comandos de desenho do HTML5 Canvas.

Cinemáticos

Como nos primeiros jogos, as animações se restringiam a demonstrações curtas do gameplay em loop, enquanto o jogo não começa. Para versões futuras é desejável que sejam introduzidas cenas de transição, briefing de missões, finais do jogo, e outros.

Vídeo

O jogo não usará arquivos de vídeo, mas estudaremos a possibilidade de exibir e salvar o replay e salvar print screens.

Som e Música

Elemento essencial em todo videogame, a música e efeitos sonoros devem corresponder à estética e crie a ambientação correspondente com a fase.

Metas Gerais

Sons low profile eletrônicos dos games antigos formando proceduralmente dance music é a meta ideal a ser atingida.

Efeitos Sonoros

Arquivos de áudio digital e sons gerados na Web Audio API:

Música

Dance music e eletrônica em tracks de áudio. É desejável que seja incluído sequenciador interno que permita música procedural, que evolui de acordo com a cena e progresso das fases.

Jingles de eventos

Produção de vinhetas para eventos de sucesso, falha, morte, vitória, descoberta e outros.

Shell Screen

Junto com os sons de evento são apresentados textos sobrepostos à cena do jogo, indicando pontos ganhos, ganho de nível e mudança de fase.

Tema das fases

Cada mini game deve ter sua música própria.

Situações

Cada fase é inspirada em um clássico arcade ou console 8 bits.

História

Ao contrário dos jogos que apresentam uma introdução com contextualização da história e apresentação de objetivos, o jogo começa direto no personagem ambientado com música e NPCs se movimentando, convidando para a ação exploratória, coleta, desviar de obstáculo e acumular pontos, evitando sofrer danos ou esgotar os recursos e entender a dinâmica das fases com os mini jogos em gameplay diverso. É dentro das fases que pode ser apresentada alguma história ou objetivo mais elaborado.

Requisitos de Fase

Diagrama de Fases

O jogo segue uma sequência linear e apresenta fases laterais que o usuário acessa e retorna ao caminho principal.

Ordem que os elementos aparecem

.Algoritmo de geração procedural garantirá que a experiência seja diferente a cada partida. Ao atingir objetivos e pontuação, novos algoritmos são fornecidos pelo servidor do jogo.

Level Design Seeds

Testes de jogabilidade no protótipo orientarão o padrão de distribuição dos elementos do jogo e progressão das fases.

Apêndice 2 – PLANO DE NEGÓCIO EMOJIGAME

Introdução

Esta é a segunda versão, ainda em desenvolvimento e incompleta, do plano de negócios para integrar o Trabalho de Conclusão de Curso que complementa esse documento. Este documento começou a ser criado durante o curso de Empreendedorismo com a Profa. Suzana.

Sumário Executivo

Este documento apresenta o projeto de desenvolvimento do Plano de Negócio para Desenvolvimento, Publicação, Comercialização e Gestão de um Web Game com a temática dos Emoji e outras figuras do conjunto Unicode.

Deverá ser implantado, desenvolvido e gerido através de sistema com framework padrão COBIT5 a serem implantados pela gestão da Empresa Júnior da Fatec de Ribeirão Preto em parceria com a comunidade interessada em jogar, aprender e contribuir com a programação, administração e conteúdo do jogo.

Contém também uma proposta para Gamificação do desenvolvimento do próprio jogo e seu sistema de gestão.

Propõe a criação de criptomoeda própria, o EmojiCoin, que representa participação na propriedade e participação nos lucros provenientes do jogo.

Está programado para ser desenvolvido e preparado no segundo semestre de 2018 (junto com a implantação da Empresa Júnior da FATEC) para o lançamento no começo de 2019.

Visão

Para gamers e estudantes de desenvolvimento de jogos, as adoráveis e cultuadas figuras dos Emoji e todos outros caracteres do conjunto Unicode agora são a matéria prima na produção de uma nova geração de Games “modo texto”. O EMOJIGAME é uma proposta de gamificação da pesquisa e desenvolvimento colaborativo do jogo, pago com participação lastreada em criptomoeda própria.

Missão

Apresentar ao mundo a ideia e a proposta de voltarmos a produzir e nos divertir com jogos modo texto, difundindo a riqueza do Unicode e nos valendo da notoriedade dos Emoji, fazendo do EmojiGame um ambiente online de concentração de diversão e aprendizado para jogadores, desenvolvedores, estudantes e investidores.

Fazer da Empresa Júnior da Fatec Ribeirão Preto uma referência de valor como produtora, gestora de jogos digitais e referência como academia para estudantes da indústria de jogos.

Oportunidade

“Os Jogos eletrônicos ou games estão inseridos em todas as camadas da sociedade, idealizados e criados para diversos propósitos tais como treinamento, educação, entretenimento e publicidade, não excluindo qualquer faixa etária ou gênero.

(...)Em 2016 a indústria de games movimentou quase US$ 100 bilhões e cresceu 21% em relação ao ano anterior, e o Brasil foi o 4o maior mercado consumidor de games, com mais de 60 milhões de usuários (Newzoo, 2016). Entretanto, as empresas brasileiras de desenvolvimento não acompanharam o crescimento, e ficaram na 11a posição mundial em receitas geradas, com US$ 1,5 bilhões. Ainda há muito potencial e mercado a ser explorado. Entre os polos de maior destaque da indústria de jogos digitais estão os de São Paulo e Campinas, que cresceram à sombra da USP e da Unicamp, o Porto Digital de Recife, que abriga perto de 120 companhias de software, e outros espalhados por Rio de Janeiro, Porto Alegre, Florianópolis, Belo Horizonte, Brasília e região de Londrina e Maringá. Com a ampliação de cursos técnicos e superiores especializados em desenvolvimento de games, novos polos surgem.” (Cartilha sobre produtora de games do Sebrae)

Existem centenas de iniciativas de se empregar os Emoji em jogos, mas nossa iniciativa é a primeira no uso direto dos caracteres texto renderizados na tela do jogo, sem o uso de *sprites*, sem a necessidade de produção artística em softwares de edição de imagens, tendo como objetivo incentivar e facilitar a produção de jogos para estudantes de programação e fazendo da tecnologia uma via de acesso ao desenvolvimento de jogos.

Poucas (ou nenhuma?) produtoras de jogos na região.

Propostas de Valor

Temática mundialmente popular dos EMOJI desenvolvida de forma inédita com autoria própria, detentor de todos direitos de criação e sem gastos com licenciamentos.

Um jogo produzido pela Empresa Júnior da Fatec conta com capital intelectual de Analistas Desenvolvedores e Gestores de Negócios, o apoio e infraestrutura do grupo de Inovação do Centro Paula Souza, a orientação e tutoria do Corpo Docente da Fatec Ribeirão Preto, poderá firmar parceria com cursos de Desenvolvimento de Jogos de outras unidades da Fatec e outras instituições de ensino.

Desenvolvido com metodologias ágeis, melhores práticas em engenharia de software, gerenciamento de banco de dados, segurança da informação, padrões de projeto e qualidade, ferramentas de suporte informatizados, em conformidade com os padrões dos frameworks de gestão de tecnologia da informação (ITIL, Cobit,…) promovendo redução de risco.

Convida interessados em criação, produção e comercialização de jogos eletrônicos para colaborar e fazer parte do empreendimento, disseminando a tecnologia e o capital intelectual, adquirindo tecnologia inovadora em troca de participação e fornecendo a tecnologia adquirida através da assinatura da conta

estudante/desenvolvedor. Diversifica o capital humano com a participação de desenvolvedores de jogos, músicos, escritores de conteúdo, profissionais de marketing e outros.

Mercado

"De 2005 a 2012 o mercado foi dominado pelas grandes plataformas de consoles como Xbox, Playstation e Nintendo, que permitiram o desenvolvimento de jogos muito complexos e com orçamentos restritivos à nascente indústria brasileira de jogos. Além disso, os modelos de comercialização não permitiam a inserção das pequenas empresas nacionais, restringindo-se a grandes companhias americanas e japonesas. A partir de 2010, com a maior facilidade de distribuição e de desenvolvimento, com jogos em mobile (como smartphones e tablets) e web, o cenário mudou favoravelmente.

Segundo a Newzoo (2016), o mercado mundial de consoles responde por 31% com crescimento 2,2% ao ano; os jogos para PC representam 32% e crescem 2,1% ao ano; o mobile é o maior interesse dos consumidores, com 37% e crescimento anual de 21,3%. A previsão é que o mercado mundial cresça de quase US$ 100 bilhões em 2016 para mais de US$ 118 bilhões em 2019, e as plataformas mobiles representando 46%.

Segundo estudo do BNDES (2014), há cerca de 200 empresas formalizadas no segmento de games, com faturamento médio de 75% delas até R$ 240 mil, 21% delas entre R$ 240 mil e R$ 2,4 milhões. Apenas 4% superaram os R$ 2,4 milhões. Outro dado relevante é que 96% das empresas exportavam, sendo que 75% delas o faziam continuamente e 25% eventualmente, e 70% das receitas foram provenientes de exportação. O que facilita a exportação é a internet e a possibilidade de se disponibilizar os jogos em distintas plataformas e lojas digitais, em qualquer lugar do mundo. A questão de idioma é relativamente fácil de ser solucionada no desenvolvimento do game. Segundo levantamento do Portal Exame, a média de funcionários nesses estúdios é de 8,5 pessoas, incluindo os sócios- fundadores."

Nichos de Mercado (Público alvo)

É um produto para todas as idades a partir dos 10 anos. Comunidade gamer interessada em webgame casual multiplataforma. Alunos interessados em aprender programação e produção de jogos digitais. Desenvolvedores de jogos.

Estratégia de Mercado – Fonte de Recursos – Modelo de negócio

Produtos, Serviços e Criptomoeda própria.

Freemium Game, livre para jogar, mas que exige a conta premium com EmojiWallet para poder guardar o dinheiro do jogo, os EmojiCoins.

Marketing para a "Mineração dos EmojiCoins", que é o desenvolvimento colaborativo do jogo, pago com EmojiCoins que representam participação real na propriedade do negócio.

Propostas de Valor – *Serious Game*

"A principal característica dos *serious games* são jogos baseados em pesquisa e desenvolvidos e também feito em universidades.
Podemos observar que pode ser aplicado no segmento da gestão corporativa e desenvolvimento de software com conteúdos alinhados as propostas de valor com: conexão social,aprendizagem, desenvolvimento profissional, prevenção, acesso à informação, recreação e experiência emotiva."

Gamificação do desenvolvimento, material para estudo disponível, implantação do Cobit5 (e ITIL) com sistema online informando fluxo de caixa, balanço, lucro e perda, e projeções.

Clientes

Comunidade gamer internacional, Estudantes e desenvolvedores de jogos.

Competidores

Outras engines e ambientes de desenvolvimento de jogos, como Unity e Unreal são imbatíveis no que se propõe, mas matam a mosca com um tiro de canhão quando são usadas na produção de jogos simples, sendo difíceis para novatos em programação, já que estudantes de programação não têm formação em artes gráficas nem em modelagem 3D, o que é uma barreira para se dedicarem à programação de jogos.

O EmojiGame é inovador em seu nicho ao possibilitar o aprendizado da programção de jogos dispensando a produção de elementos gráficos graças ao poder do HTML5 Canvas e à riqueza de conteúdo do conjunto de caracteres e símbolos Unicode.

Estratégia de Negócio – Renda

O acesso ao ambiente do jogo será sempre gratuito para qualquer um com acesso à internet poder jogar e se divertir sem compromisso algum, facilitando a popularidade, aumentando o fluxo de usuários e fazendo do ambiente do jogo um lugar de valor para anunciantes.

A proposta de gamificação da produção colaborativa do jogo, com chamada de marketing para a "Mineração da criptomoeda EmojiCoin" surfa na atual onda dos bitcoins e outras criptomoedas, atraindo atenção da mídia e partes interessadas.

As principais fontes de renda serão:
. as assinaturas premium, com acesso à EmojiWallet.
. assinaturas estudante, com acesso aos cursos de programação, produção e gestão de jogos digitais.
.assinatura pro, com EmojiWallet e acesso aos prêmios dos campeonatos.
. licença do software assistente de edição jogos, fases e personagens.
.produtos Licenciados com os Personagens (roupas, brinquedos, outros).

,propaganda no ambiente do Jogo.
.micro pagamento de itens virtuais.
.dados da base de usuários.
.câmbio dos EmojiCoins.

Canal de Vendas

As diversas categorias de assinaturas, licenças e produtos associados serão comercializadas diretamente no site do jogo, que deve contar com canais de comunicação com o suporte ao usuário

Organização e Operação – Dados dos Empreendedores

Todo o projeto será dirigido e operacionalizado pela Empresa Júnior da Fatec Ribeirão Preto, contando com a estrutura das instalações, orientação e tutoria dos professores da Fatec e apoio da Agência de Inovação do Centro Paula Souza.

Gerenciamento

Segundo o manual de produção de games do Sebrae ‘o maior motivo de insucesso das empresas de games está na baixa capacidade de gestão. O Sebrae pode apoiar o desenvolvimento empresarial por meio de diversas soluções, como capacitações e consultorias;” Os alunos do curso de graduação tecnológica de Gestão de Negócios e Inovação estão capacitados para a administração do empreendimento e contam com apoio e parceria com o Sebrae-SP.

Os alunos do curso de tecnologia em Análise e Desenvolvimento de sistemas da Fatec são capacitados com sólidos conhecimentos em Engenharia de Software, Banco de Dados, Programação Orientada a Objetos, Programação Web, Gestão de Projetos, Empreendedorismo e vários outros conteúdos para elaborar um sistema de Gestão Integrada automatizado com recursos de Robotic Process Automation, auditável nos padrões COBIT5 e provido de Business Intelligence

Empresa Júnior Fatec Ribeirão Preto com os frameworks ITIL e Cobit
Competências e Desafios

Protótipo funcional publicado em emojigame.azurewebsites.net

Desafios

Desafios já identificados podem ser elencados:
Melhorar a performance em dispositivos mobile.
Limitações dos navegadores e sistemas operacionais.
Desenvolver gameplay, fases e conteúdo das missões.
Desenvolvimento do sistema monetário EmojiCoin
Migração da tecnologia para consoles

Finanças

Recursos Necessários, Gastos (investimentos, custos e despesas)

Pesquisa e Desenvolvimento

Nas primeiras fases do projeto o desenvolvimento foi realizado pelo autor do projeto de forma independente. Com a tecnologia testada e a proposta elaborada, continua o desenvolvimento junto com o desenho do sistema de desenvolvimento colaborativo gamificado, com o projeto e tecnologia disponibilizados para que possamos nos desenvolver, fazer contatos, criar valor e obter ativos.

Sede – Instalações físicas

A Fatec oferece a estrutura e acompanhamento do projeto através de sua Empresa Júnior, o que corta gastos fixos. Os desenvolvedores trabalharão home office ou trarão seus próprios equipamentos (byod).

Propriedade intelectual

A ideia original elaborada nos laboratórios da Fatec é de autoria própria e estará formalmente cedida por contrato para ser explorada pela Empresa Júnior, o que evita gastos com licenciamentos e trará ganhos com os produtos relacionados ao jogo e seus personagens.

Capital Humano

Membros da Empresa Júnior, alunos da Fatec envolvidos com o projeto e colaboradores externos.

Financiamento

EmojiCoins oferecidos como títulos de participação.

Hospedagem, recurso chave do negócio.

O principal investimento necessário ao negócio é a implantação ou contratação de servidor, prevendo a escalabilidade.

Marketing

O plano de marketing é inspirado no Marketing Mix do Kotler.: produtos, preço, promoção e praça.

Preço:

Freemium Game, com micropagamentos para itens.
Conta Premium, com EmojiWallet para guardar seus EmojiCoins: $ 10,00
Conta Master, com EmojiWallet, ferramentas de desenvolvimento e administração do jogo $ 30,00
EmojiCoin: cotação variável

Produto:
Ambiente de Jogos multiplayer com comunidade de desenvolvedores, gamificação do desenvolvimento colaborativo com criptomoeda própria.

Ponto de Venda:
Webgame multiplataforma internacionalizado (traduzido automaticamente para diversos idiomas)

Promoção:
Chamada para a mineração dos Emojicoins através da Pesquisa e Desenvolvimento Gamificado do EmojiGame com criação e divulgação de Teaser Vídeo divulgado em grupos de criadores de jogos e relacionados nas redes sociais.

Plano Financeiro
Apoio Financeiro
Patrocinadores anunciantes.
Captação de recursos“há diversos editais específicos para o mercado de games, como da Ancine, SPCine e BNDES.”
Estimativa de investimento inicial
Adquirir um computador e uma conexão dedicada. R$ 5000,00
Estimativa de custo fixo
Pro labore dos operadores do sistema. R$ 400,00
Manutenção da conexão com a Internet. R$ 100,00
Estimativa de custo variável
Campanhas de marketing R$?
Estimativa de receita
Vender 30 contas premium e 10 contas master por mês US$ 600,00
Valores
Fluxo de Caixa Descontado (Discounted Cash Flow – DCF)
FCF = EBIT(1-T) + Depreciação – Investimento de Capital – Investimentos em capital
de giro
Sendo,
FCF = Fluxo de Caixa Livre
Custo Médio de Capital Ponderado (Weighted Average Cost of Capital - WACC)
EBIT = Lucros antes de juros e imposto de renda
T = Alíquota do imposto de renda
Indicadores de Viabilidade
Lucratividade
Rentabilidade
Prazo de retorno do investimento
Ponto de Equilíbrio

Estimativa de tributos a pagar

‘O segmento de PRODUÇÃO DE GAMES, assim entendido pela CNAE/IBGE (Classificação Nacional de Atividades Econômicas) 6203-1/00 como a atividade de desenvolvimento de sistemas ou programas de computador que não permitem customizações, tais como: jogos de computador para todas as plataformas, poderá optar pelo SIMPLES Nacional - Regime Especial Unificado de Arrecadação de Tributos
e Contribuições devidos pelas ME (Microempresas) e EPP (Empresas de Pequeno Porte), instituído pela Lei Complementar nº 123/2006, desde que a receita bruta anual
de sua atividade não ultrapasse a R$ 360.000,00 (trezentos e sessenta mil reais) para
micro empresa, R$ 3.600.000,00 (três milhões e seiscentos mil reais) para empresa de
pequeno porte e respeitando os demais requisitos previstos na Lei.

Nesse regime, o empreendedor poderá recolher os seguintes tributos e contribuições, por meio de apenas um documento fiscal – o DAS (Documento de Arrecadação do Simples Nacional), que é gerado no Portal do SIMPLES Nacional (http://www8.receita.fazenda.gov.br/SimplesNacional/):

• IRPJ (imposto de renda da pessoa jurídica);
• CSLL (contribuição social sobre o lucro);
• PIS (programa de integração social);
• COFINS (contribuição para o financiamento da seguridade social);
• ISSQN (imposto sobre serviços de qualquer natureza);
• INSS (contribuição para a Seguridade Social relativa a parte patronal).

Conforme a Lei Complementar nº 123/2006, as alíquotas do SIMPLES Nacional, para esse ramo de atividade, variam de 6% a 17,42%, dependendo da receita bruta auferida pelo negócio. No caso de início de atividade no próprio ano-calendário da opção pelo SIMPLES Nacional, para efeito de determinação da alíquota no primeiro mês de atividade, os valores de receita bruta acumulada devem ser proporcionais ao número de meses de atividade no período.”

Projeção de fluxo de caixa

Payback

Valor presente líquido

Bibliografia

Como montar uma produtora de games – Sebrae-SP

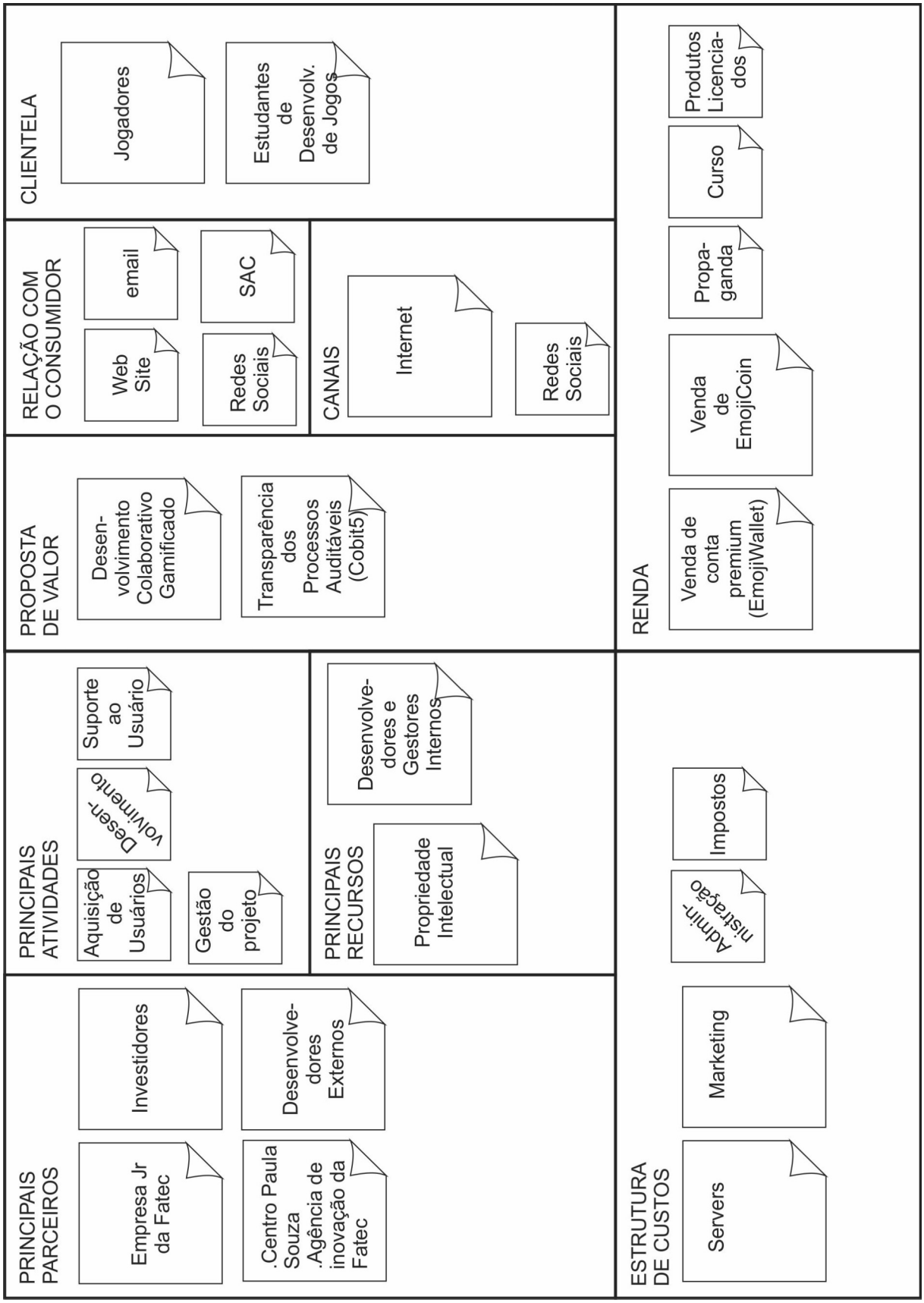
file:///C:/Users/caode/Downloads/Produ%C3%A7%C3%A3o%20de%20games%20(1).pdf

http://www.abragames.org/manual-de-questotildees-juriacutedicas-de-empresas-de-games.html

Como elaborar um Plano de Negócio – SEBRAE

Manual do Plano de Negócio - SEBRAE

Apêndice 3 – Modelo Canvas para o EMOJIGAME



Anexo 1 - Edital de especificação de projeto do Laboratório de Engenharia de Software

“CENTRO PAULA SOUZA FACULDADE DE TECNOLOGIA DE RIBEIRÃO PRETO CURSOS DE TECNOLOGIA EM: ANÁLISE E DESENVOLVIMENTO DE SISTEMAS LABORATÓRIO DE ENGENHARIA DE SOFTWARE EDITAL 02 / 2017 Para a execução do projeto da disciplina Laboratório de Engenharia de Software do curso de Análise e Desenvolvimento de Sistemas da Fatec Ribeirão Preto, o professor responsável pela referida disciplina torna público o presente Edital de NORMAS PARA O DESENVOLVIMENTO DO PROJETO DE LABORATÓRIO DE ENGENHARIA DE SOFTWARE - MANHÃ - DO SEGUNDO SEMESTRE DE 2017. I – OBJETIVOS DO PROJETO: 1. Este projeto tem por objetivo, de acordo com o plano de ensino da disciplina: a. Especificar, projetar e implementar um software aplicando conhecimentos de engenharia de software, programação e gerência de projetos, tendo como base a ementa da disciplina, a saber: i. Desenvolvimento de um software utilizando os conhecimentos adquiridos ao longo do curso. A elaboração deve abordar as disciplinas de requisitos, análise e projeto, implementação, implantação e gerência de projetos. O processo de desenvolvimento, assim como a técnica fica a critério de acordo entre professor e aluno. II – MATERIAIS E MÉTODOS: 1. Das tecnologias utilizadas: a. entende-se por tecnologia: i. ferramentas para análise, projeto e gerência de projeto de software, frameworks, API (Application Program Interface), bibliotecas, bancos de dados, compiladores, servidores e linguagens de programação. b. caberá ao grupo a escolha das tecnologias utilizadas no desenvolvimento do projeto. i. sugere-se fortemente que sejam escolhidas tecnologias abordadas no curso de ADS. ii. é de responsabilidade do grupo qualquer problema advindo da escolha de qualquer tecnologia. 2. Da gerência de projetos: a. cada grupo deverá adotar uma ferramenta para gerenciamento de projetos que contemple o cronograma de desenvolvimento em tarefas atrelada aos membros da equipe, bem como o tempo e duração de cada uma. III – FORMAÇÃO DOS GRUPOS 1. Da composição: a. Cada grupo deverá possuir o mínimo de três e o máximo de cinco alunos, e deverá, obrigatoriamente, possuir um nome significativo para uma empresa de desenvolvimento de sistemas. 2. Dos papéis: a. Cada grupo deverá escolher um gerente de projeto, o qual terá as seguintes atribuições: i. gerenciar o projeto, mantendo e verificando as tarefas constantes na ferramenta de gerência de projetos. ii. gerenciar o cronograma adequadamente juntamente com seus membros. iii. alinhar as atividades de forma a sincronizar as atividades interdependentes entre os membros do grupo. iv. responder gerencialmente e burocraticamente pelo grupo. ATENÇÃO: apesar da liderança exercida pelo gerente todos os participantes são igualmente responsáveis pelo desenvolvimento do projeto. b. Outros papéis: i. Demais papéis serão definidos de acordo com a demanda do projeto a partir dos alinhamento realizados entre os membros da equipe. 3. Da formação: a. Não serão aceitas mudanças nos componentes do grupo a partir de sua formação. b. Quaisquer demandas deverão ser apresentadas ao docente responsável pela disciplina, o qual decidirá os procedimentos adequados, se houver. IV – DA AVALIAÇÃO DO PROJETO 1. Das notas: a. a nota será calculada de acordo com a seguinte formula: i. onde: 1. NA: nota da apresentação atribuído ao grupo no dia da apresentação do projeto. Cada membro da banca convidada para avaliar o trabalho dará uma nota ao grupo seguindo critérios estabelecidos. 2. NI: nota individual para cada aluno e será

atribuída pelo docente responsável pela disciplina a partir do acompanhamento de cada um estudantes em seus respectivos grupos. 3. NF: nota final da disciplina. ATENÇÃO: será aprovado o estudante que obtiver . 2. Das atividades: a. As principais atividades constantes de avaliação estão especificadas abaixo, e serão datadas na seção V - CRONOGRAMA. i. Formação de grupos. ii. Especificação do domínio/problema. iii. Documento de requisitos. iv. MER, Diagrama de casos de uso e classes. v. Entrega final. vi. Apresentação. b. O professor poderá requerer apresentações à quaisquer grupos bem como reuniões de acompanhamento para fins de alinhamento ou avaliação. ATENÇÃO: O não cumprimento das datas de entrega, ou seja, atrasos, serão penalizados com a perda de pontos. 3. Da banca: a. a avaliação final na apresentação será feita por uma banca composta por professores do curso e profissionais convidados. b. a banca poderá arguir o grupo com o intuito a sanar eventuais dúvidas quanto ao desenvolvimento. V- DO CRONOGRAMA 1. O cronograma de atividades a serem executadas durante as aulas da disciplina de PI está apresentado a seguir: Agosto: 08: Entrega e explicação do Edital. Esclarecimento de dúvidas 15: Acompanhamento do projeto com os grupos. NF = NA + NI 2 NF ≥ 6,0 22: Fechamento da formação dos grupos. 29: Acompanhamento do projeto com os grupos. Setembro: 05: Acompanhamento do projeto com os grupos. 12: Especificação do domínio/problema. 19: Acompanhamento do projeto com os grupos. 26: Acompanhamento do projeto com os grupos. Outubro: 03: Documento de requisitos. 10: Acompanhamento do projeto com os grupos. 17: Acompanhamento do projeto com os grupos. 24: Modelo Entidade-Relacionamento e Diagrama de Classes 31: Acompanhamento do projeto com os grupos. Novembro: 07: Acompanhamento do projeto com os grupos. 14: Acompanhamento do projeto com os grupos. 21: Acompanhamento do projeto com os grupos. 28: Acompanhamento do projeto com os grupos. Dezembro: 05: Entrega final (Sistema implementado) 09 (Sábado): Apresentação do projeto. 12: Finalização da disciplina. VI – DO RELATÓRIO FINAL 1. Um relatório final deverá ser entregue no dia da apresentação final contendo os seguintes elementos: a. Capa: contendo o nome da empresa e os nomes dos integrantes do grupo; b. MER, diagramas de caso de uso e diagrama de classes; c. Descrição das funcionalidades: devem ser apresentadas as telas e a descrição das funcionalidades referentes a cada uma; d. Anexo: o código fonte completo e documentado. VII - DA APRESENTAÇÃO ORAL 1. Todos os integrantes do grupo deverão comparecer na data especificada no cronograma para a apresentação oral do projeto. O não comparecimento na apresentação oral implica em reprova, quer seja do aluno (no caso da falta de um aluno), quer seja do grupo (no caso da falta de todos os integrantes do grupo). 2. A apresentação deve ser feita em powerpoint contendo: nome do grupo e seus integrantes, descrição geral das funcionalidades do sistema e seus diferenciais. O objetivo da apresentação é reproduzir um cenário de apresentação para possível venda do produto criado por cada grupo e, neste caso, o grupo deve usar a criatividade para vender o seu referido produto feito. 3. O tempo disponível para cada grupo será divulgado com antecedência de no máximo 15 dias e será estabelecido de acordo com as condições de local e data a serem verificadas próximo da data de apresentação. 4. A data inicialmente especificada no cronograma poderá sofrer alterações caso haja necessidade devidamente identificada pelo professor responsável. VIII - DO CÓDIGO DE CONDUTA Todos os estudantes se comprometem com o código de conduta abaixo. 1. Em avaliações não aceite ou peça assistência de qualquer natureza. As avaliações individuais, como o próprio nome diz, não permitem a interferência de agentes externos na resolução do

problema. São considerados agentes externos, ou assistência de qualquer natureza, orientações, discussões, apoio em qualquer estratégia e/ou codificação que não seja planejada pelo estudante em questão. Além disso, aceitar/pedir ajuda virtual ou não na remoção de eventuais bugs configura uma violação à esse item. 2. Não compartilhe seu código com outros estudantes. Não entregue seu código para nenhum estudante que possa vir a pedi-lo, e não peça o código de outros estudantes. Semelhantemente, não discuta estratégias algorítmicas que impliquem em códigos semelhantes. Discuta ideias com muito cuidado para não se colocar em situação de risco na avaliação. 3. Não procure e/ou olhe soluções de outros estudantes ou pessoas, seja na web ou não. A proibição de olhar eventuais soluções de codificação são muito importantes e pertinentes à qualquer curso de programação. Desenvolver bons enunciados de exercícios, trabalho e provas leva muito tempo. Quando um novo enunciado é criado, problemas aparecem fazendo com que o mesmo seja adequadamente ajustado. Sendo assim, através dos anos os enunciados são reutilizados com algumas alterações que visam a melhoria do processo ensino aprendizagem. Usar soluções de outros, seja do semestre atual ou não, ou mesmo da web, configura uma prática perigosa, uma vez que entre um semestre e outro os enunciados apresentam modificações entre si, sendo que a solução apresentada em um semestre pode não ser adequada ao próximo. Além disso é uma prática que fere o código de conduta e não será tolerada. 4. Esteja preparado para explicar qualquer código que você submeteu como seu. Tome cuidado ao requisitar qualquer tipo de ajuda, pois, como o objetivo de um enunciado é verificar seu desempenho, e uma vez que o professor esteja acompanhando o andamento dos estudantes no semestre, você pode ser requisitado a explicar seu código, com o intuito de esclarecer eventuais diferenças de desempenho apresentado na avaliação e nas aulas. Não pense que qualquer trecho de código, por menor que seja, funcionará magicamente na sua solução sem que você tenha que entendê-lo satisfatoriamente. Muitos estudantes procuram adotar medidas para disfarçar um código alheio como seu próprio código através da mudança de nomes de variáveis e funções, mera alteração de estruturas de repetição e condicionais, expressões lógicas e comentários. Não perca seu tempo fazendo isso. O professor possui métodos eficazes para detecção de plágio, alguns desses métodos são advindos da experiência no ensino e outros são automatizados. Por fim, o professor em questão observará os itens levantados no código de conduta com o intuito de detectar violações, e assim aplicará as devidas sanções aos envolvidos. IV - OUTROS Quaisquer outras questões não abordadas nesse edital deverão ser decididas pelo professor responsável pela disciplina. Ribeirão Preto, 07 de agosto de 2017. _____ Lucas Baggio Figueira De acordo: Nome do grupo: _____

Integrante: _____

Integrante: _____

Integrante: _____

Integrante: _____

Integrante: _____

Integrante: _____”